

计算机组成原理

王鸿鹏

计算机科学与技术学院

wanghp@hit.edu.cn

第七章 输入输出 (Input/Output) 系统

一. 概述

二. I/O接口

三. 程序查询方式

四. 程序中断方式

五. DMA方式

一、概述

- 输入输出系统的发展概况
- 输入输出系统的组成
- 输入输出设备与主机的联系方式
- 输入输出设备与主机信息传送的控制方式

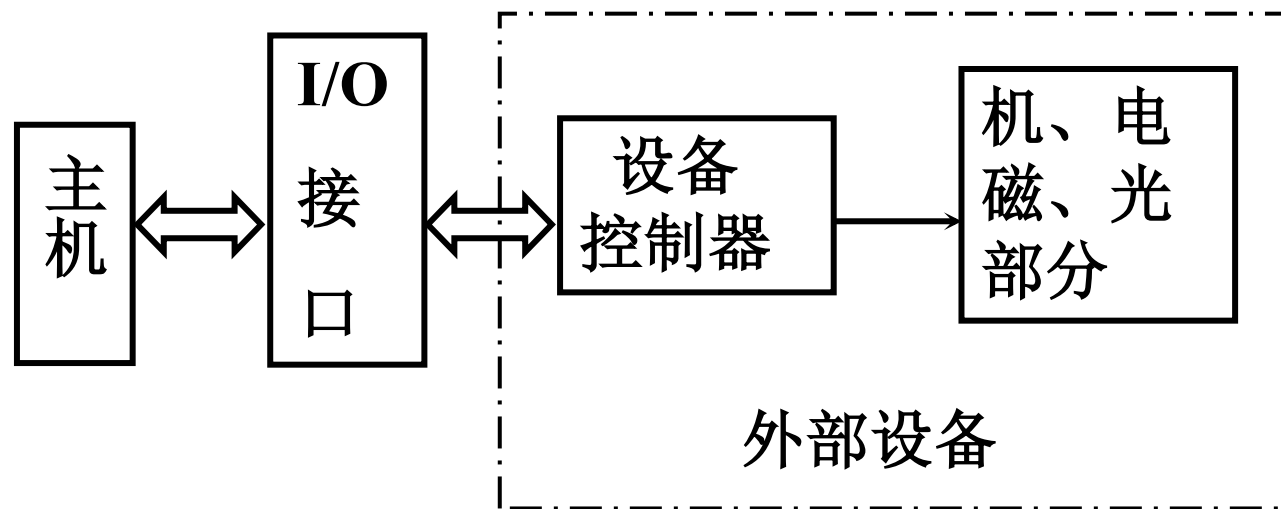
什么是输入输出系统？

- 输入输出（**Input/Output**）系统：外部设备、接口部件、总线以及相应的管理软件的统称，简称**I/O系统**。
 - 完成计算机内部二进制信息与外部多种信息形式间的交流
 - 保证CPU能够正确选择I/O设备并实现对其控制，与数据传输
 - 利用缓冲、合适的数据传送方式，实现主机外设间速度匹配



外部设备

- 人机交互设备
 - 键盘、鼠标、打印机、显示器
- 计算机信息存储设备
 - 磁盘、光盘、磁带
- 机—机通信设备
 - 调制解调器等
 - 网卡等
- **A/D、D/A**：模拟/数字（数字/模拟）转换器



输入输出过程

- 输入过程

- CPU将一个地址放在地址总线上，选择设备
- **CPU等候输入设备的数据成为有效**
- CPU从数据总线读入数据

- 输出过程

- CPU将一个地址放在地址总线上，选择设备
- CPU把数据放在数据总线上；
- **输出设备认为数据有效，取走数据**

输入输出系统的发展概况

- 早期阶段

- 分散连接

- CPU 和 I/O设备 **串行** 工作

- 接口模块和 **DMA** 阶段

- 总线连接

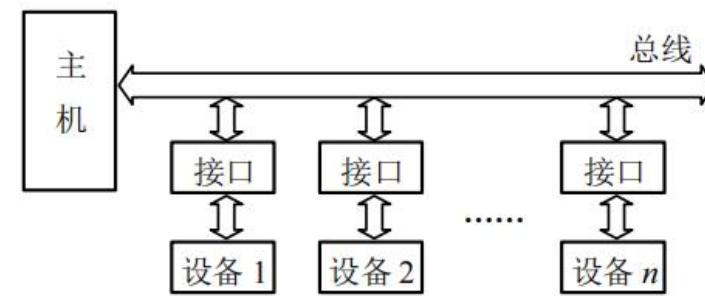
- CPU 和 I/O设备 **并行** 工作

- 具有通道结构的阶段

- 具有I/O处理机的阶段



程序查询方式



中断方式
DMA 方式



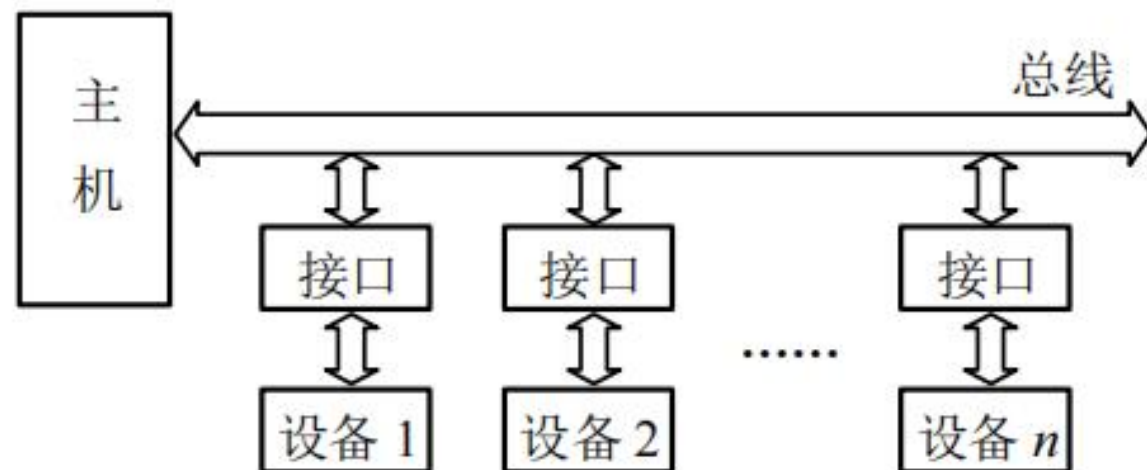
输入输出系统的组成

- I/O软件

- I/O指令 (X86 CPU 指令的一部分)
- 通道指令 通道自身的指令
 - 指出数组的首地址、传送字数、操作命令
 - 如 IBM/370 通道指令为 64 位

- I/O硬件

- I/O接口模块
- 设备控制器

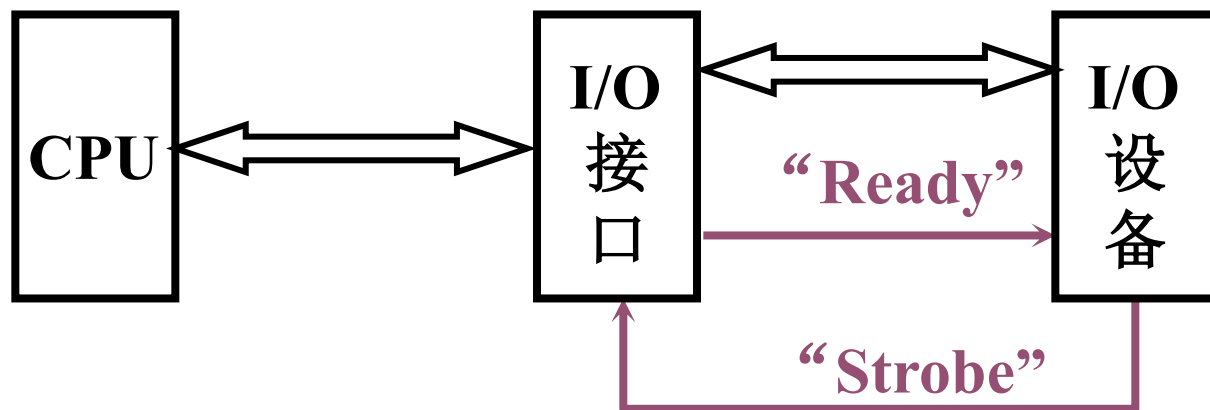


I/O 设备与主机的联系（通信）方式

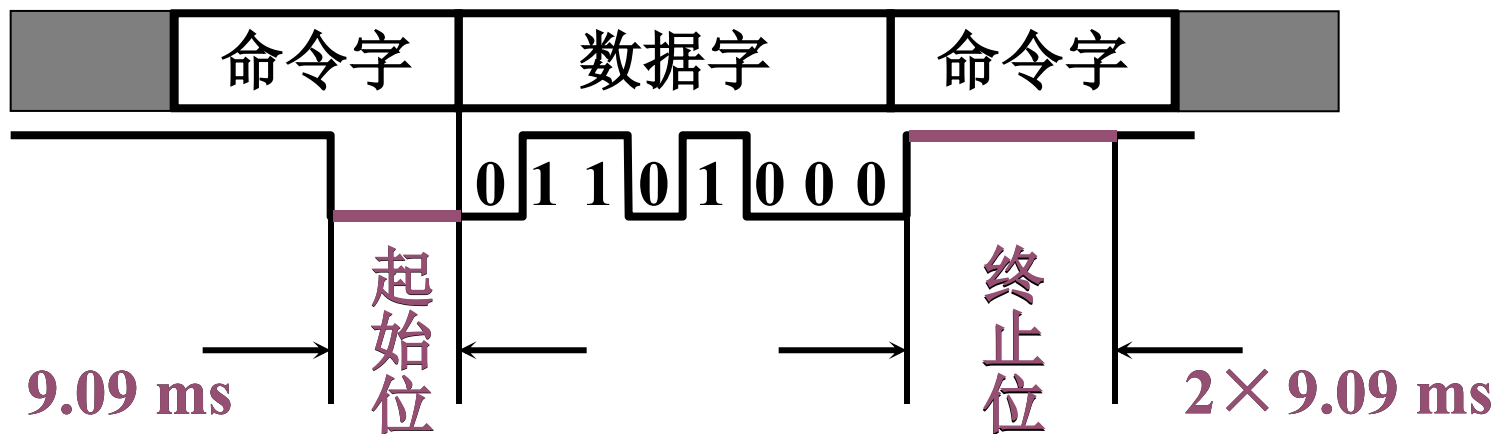
- I/O 设备编址方式
 - 统一编址：用取数、存数指令
 - 不统一编址：有专门的 I/O 指令
- 设备选址
 - 用设备选择电路识别是否被选中
- 传送方式
 - 串行、并行
- 通信方式
 - 立即响应、异步工作(采用应答信号)、同步工作(采用同步时标)

采用应答信号的异步工作

并行

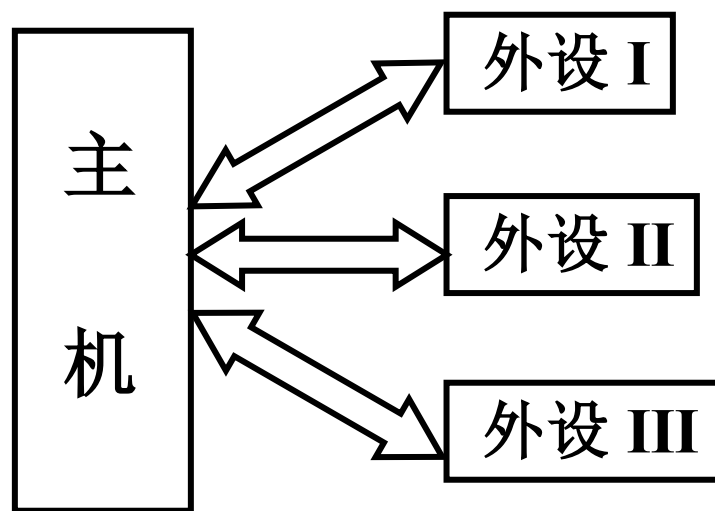


串行



I/O 设备与主机的连接方式

- 辐射式连接



每台设备都配有一套控制线路
和一组信号线

不便于增删设备

- 总线连接

便于增删设备

第七章 输入输出系统

一. 概述

二. **I/O接口**

三. 程序查询方式

四. 程序中断方式

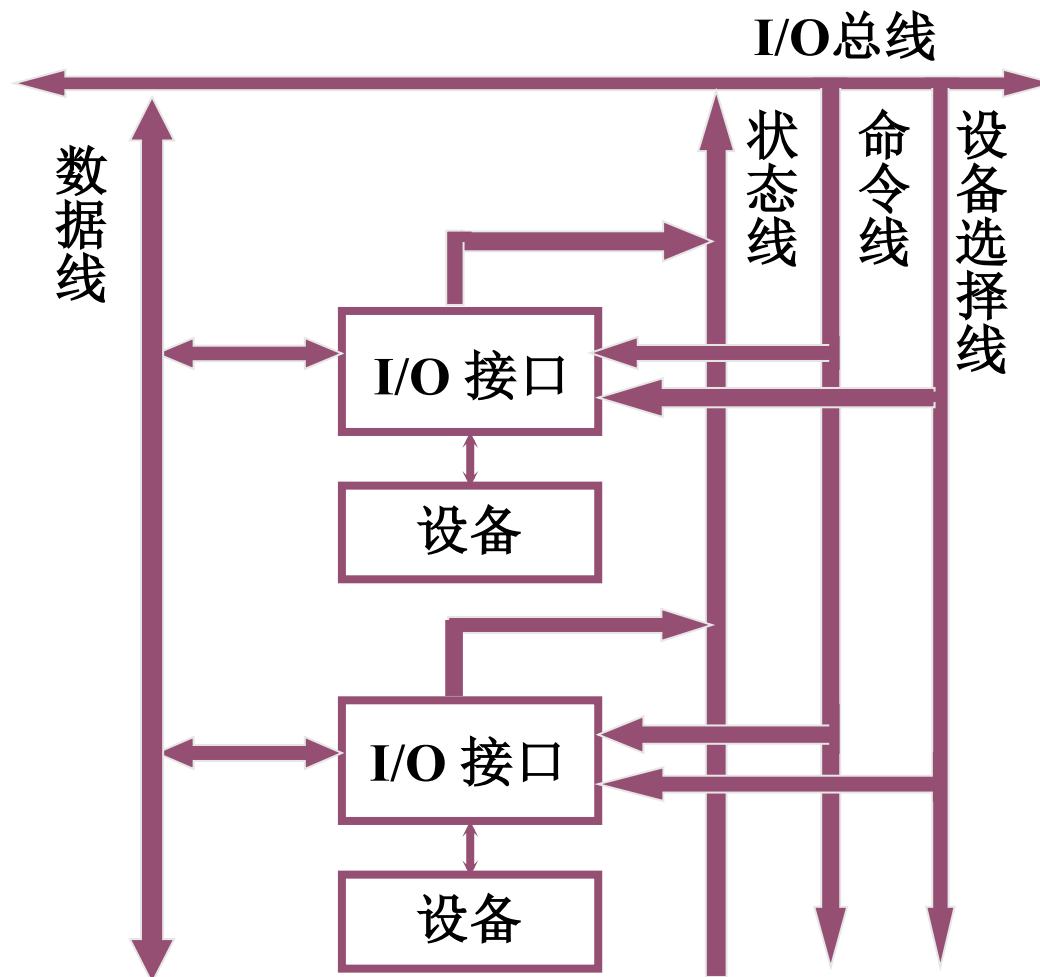
五. DMA方式

为什么要设置接口？

- 实现设备的选择
- 实现数据缓冲以达到不同外设工作速度的匹配
- 实现数据串并格式转换
- 实现电平转换
- 传送控制命令
- 反映设备的状态（“忙”、“就绪”、“中断请求”）

总线连接方式的I/O 接口电路

- 设备选择线（单向）
- 数据线（双向）
- 命令线（单向）
- 状态线（单向）



接口的功能和组成

- 选址功能
- 传送命令的功能
- 传送数据的功能

设备选择电路

命令寄存器、命令译码器

数据缓冲寄存器(Data Buffer Register)

- 反映设备状态的功能

设备状态标记

完成触发器 **D**

工作触发器 **B**

中断请求触发器 **INTR**

屏蔽触发器 **MASK**

I/O接口的基本组成



I/O接口类型

- 按数据 **传送方式** 分类

- 并行接口（可编程并行I/O接口芯片Intel 8255）、串行接口（Intel 8251）

- 按 **可编程** 分类

- 可编程接口（Intel 8255、Intel 8251）、不可编程接口（Intel 8212）

- 按 **通用性** 分类

- 通用接口(Intel 8255、Intel 8251)
- 专用接口(例如：键盘和LED显示接口Intel 8279)

- 按数据传送的 **控制方式** 分类

- 中断接口Intel 8259、DMA 接口Intel 8237

I/O设备与主机信息传送的控制方式

- 程序查询方式
- 程序中断方式
- **DMA 方式**

第七章 输入输出系统

一. 概述

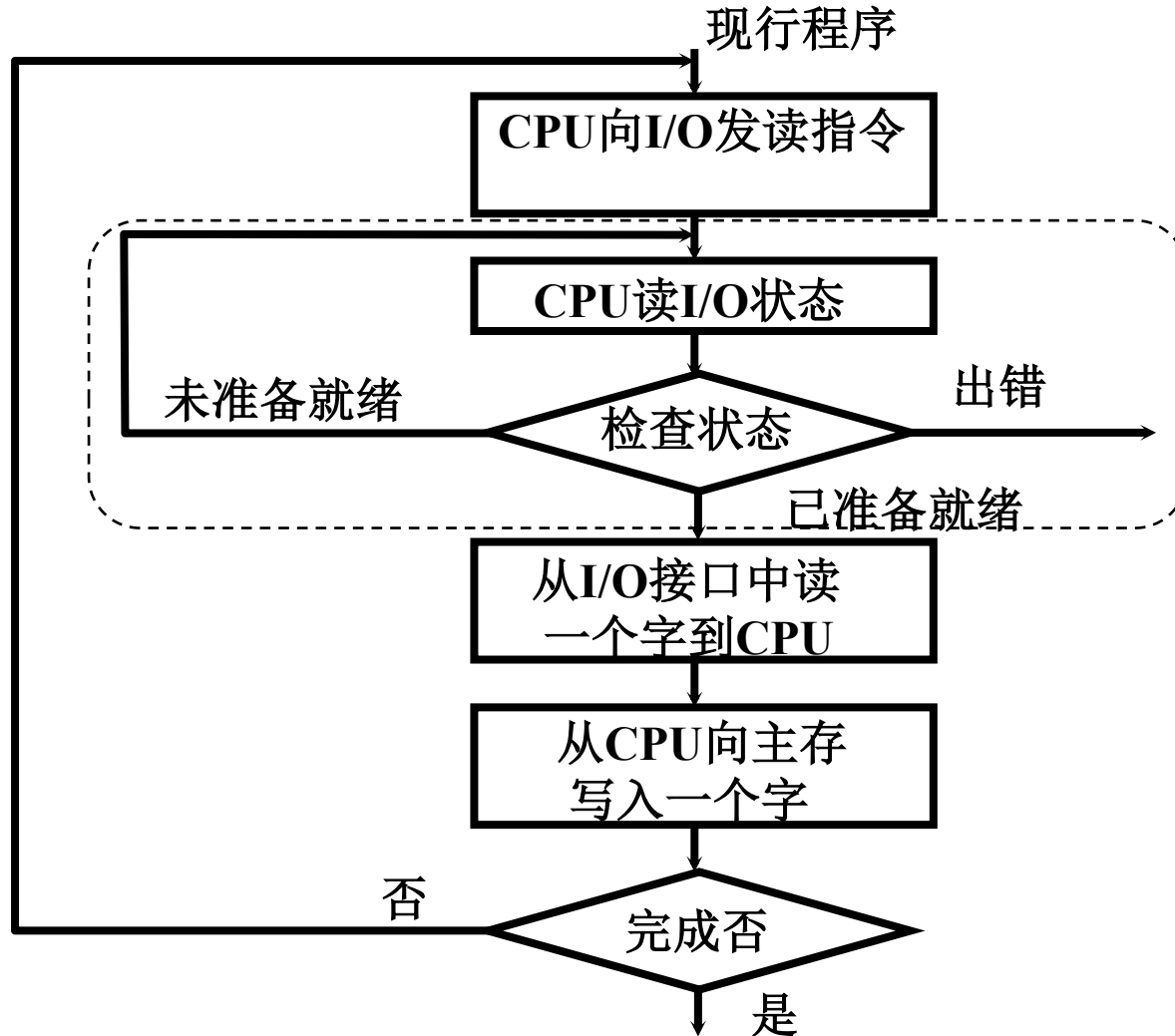
二. I/O接口

三. 程序查询方式

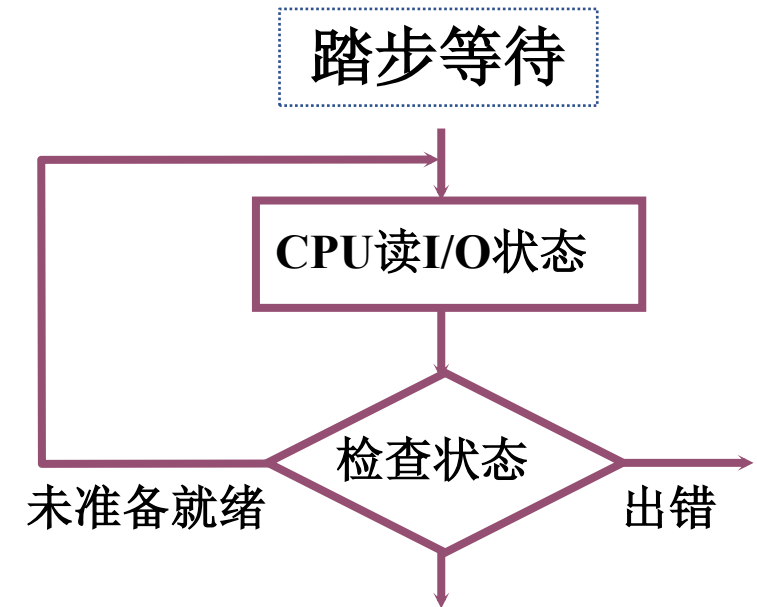
四. 程序中断方式

五. DMA方式

I/O设备与主机信息传送——程序查询方式

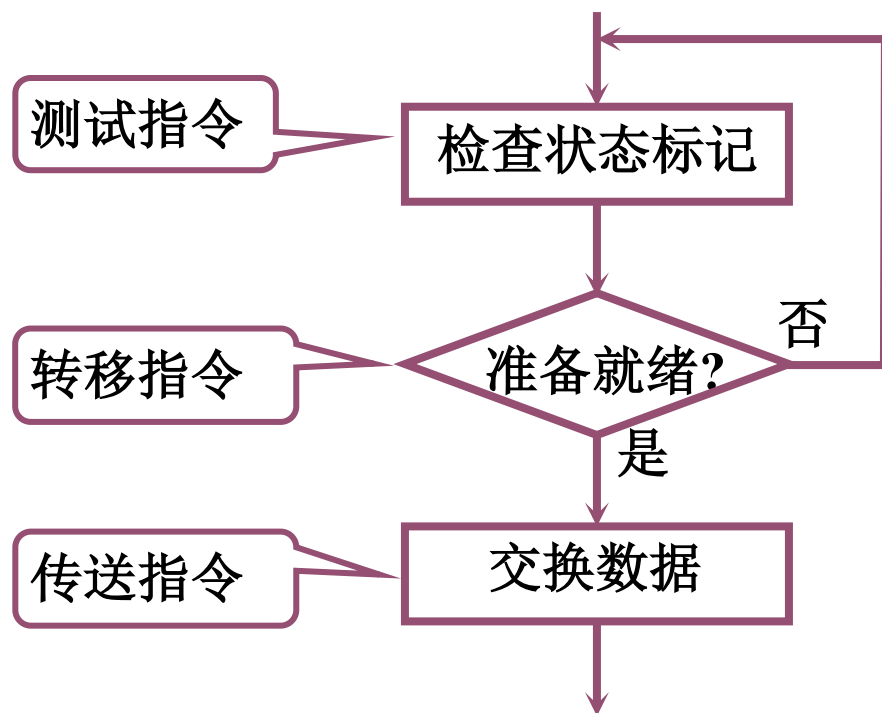


CPU 和 I/O 串行工作

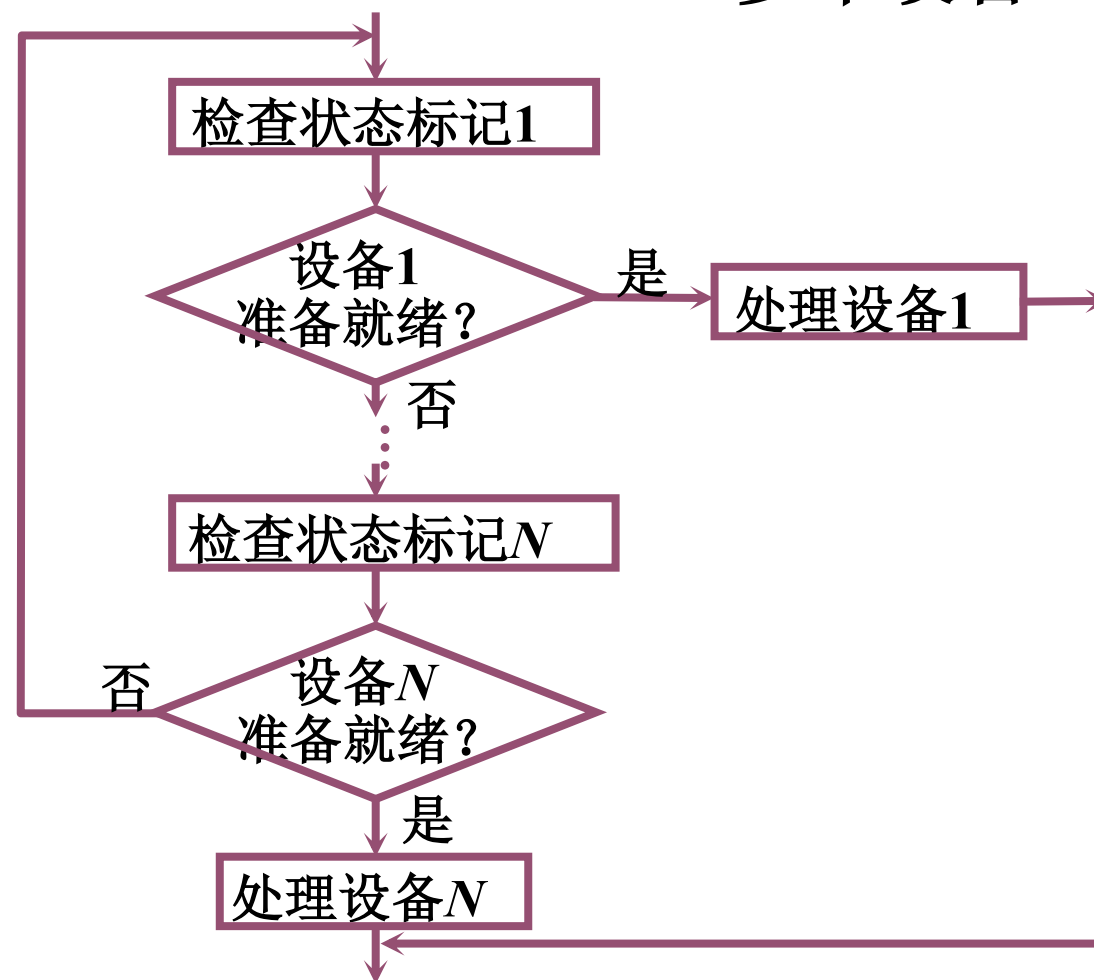


程序查询方式工作流程

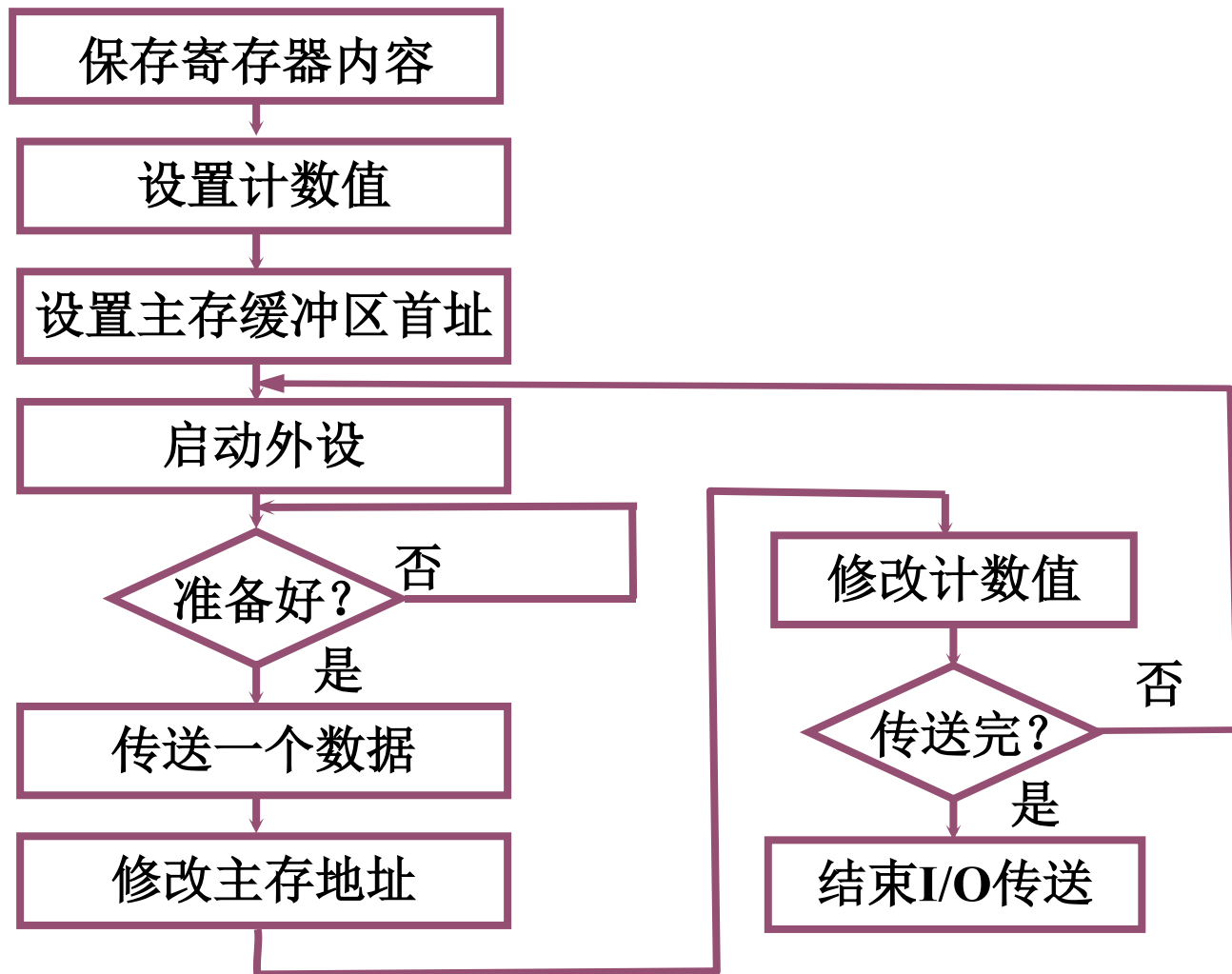
• 单个设备



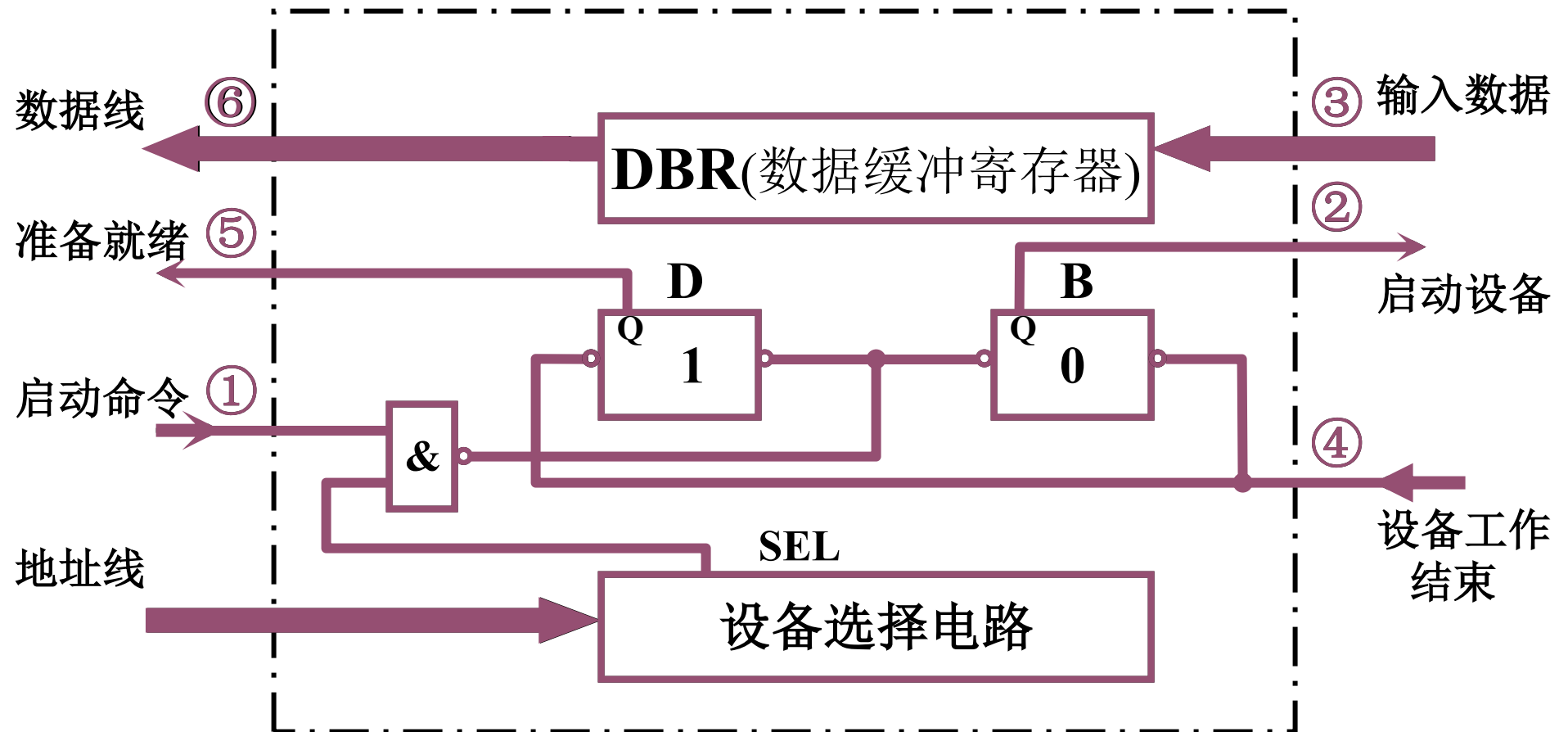
多个设备



程序查询方式软件流程图



程序查询方式的接口电路——以输入为例



例1

(假设不可考虑处理时间) CPU的时钟频率为50MHz, 每个查询操作需100个时钟周期, 每秒对鼠标查询30次, 硬盘以32位字长为单位传输数据(传输率为2MBps), 求CPU查询两个设备的时间占时钟周期的比率。得出什么结论?

解: 50MHz—>1秒有 50×10^6 个时钟周期

鼠标查询时间与时钟周期之比 $[30 \times 100 / (50 \times 10^6)] = 0.006\%$

硬盘每秒查询次数为 $2\text{MB} / 4\text{B} = 512 \times 1024 \text{次} = 52.4 \times 10^4 \text{次}$

每秒硬盘查询占 $52.4 \times 10^4 \times 100 = 52.4 \times 10^6$ 时钟周期

时钟周期占比 $[(52.4 \times 10^6) / (50 \times 10^6)] \times 100\% = 105\%$

结论: CPU与磁盘交换信息时, 不应采用程序查询方式

第七章 输入输出系统

一. 概述

二. I/O接口

三. 程序查询方式

四. 程序中**断方式**

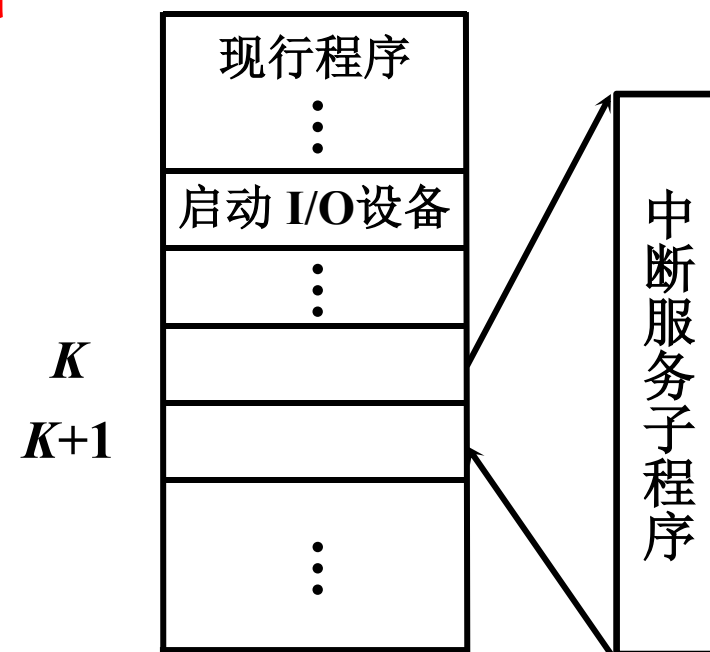
五. DMA方式

I/O设备与主机信息传送——程序中断方式

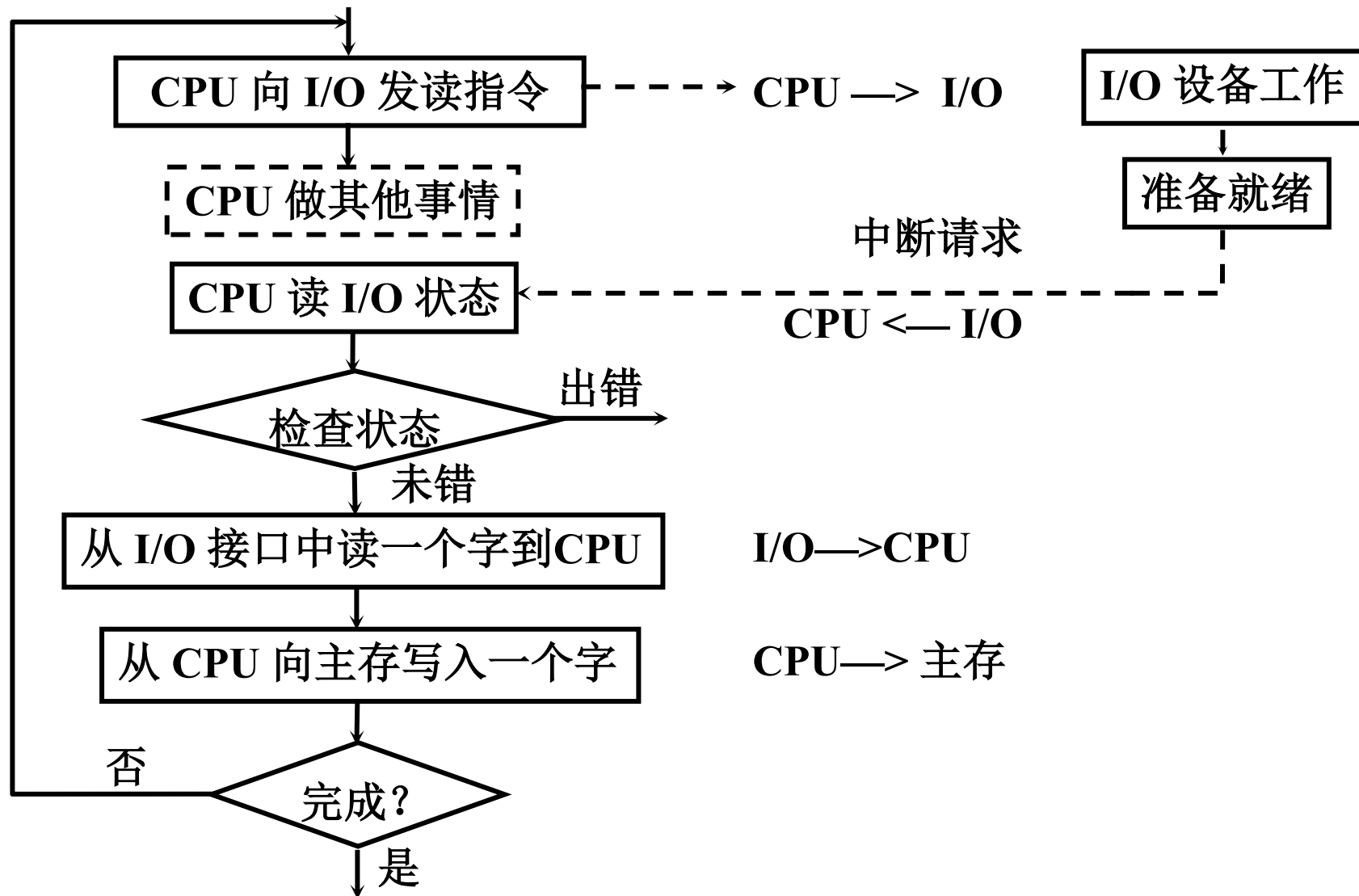
I/O 工作 { 自身准备 CPU 不查询
与主机交换信息 CPU 暂停现行程序

CPU 和 I/O 并行工作

- 无踏步等待，提高了CPU的使用效率
- 仍需占用CPU
 - 中断服务（子）程序运行时间+中断开销
- 适合**随机出现**的服务
- 需要专门的硬件



程序中中断方式工作流程

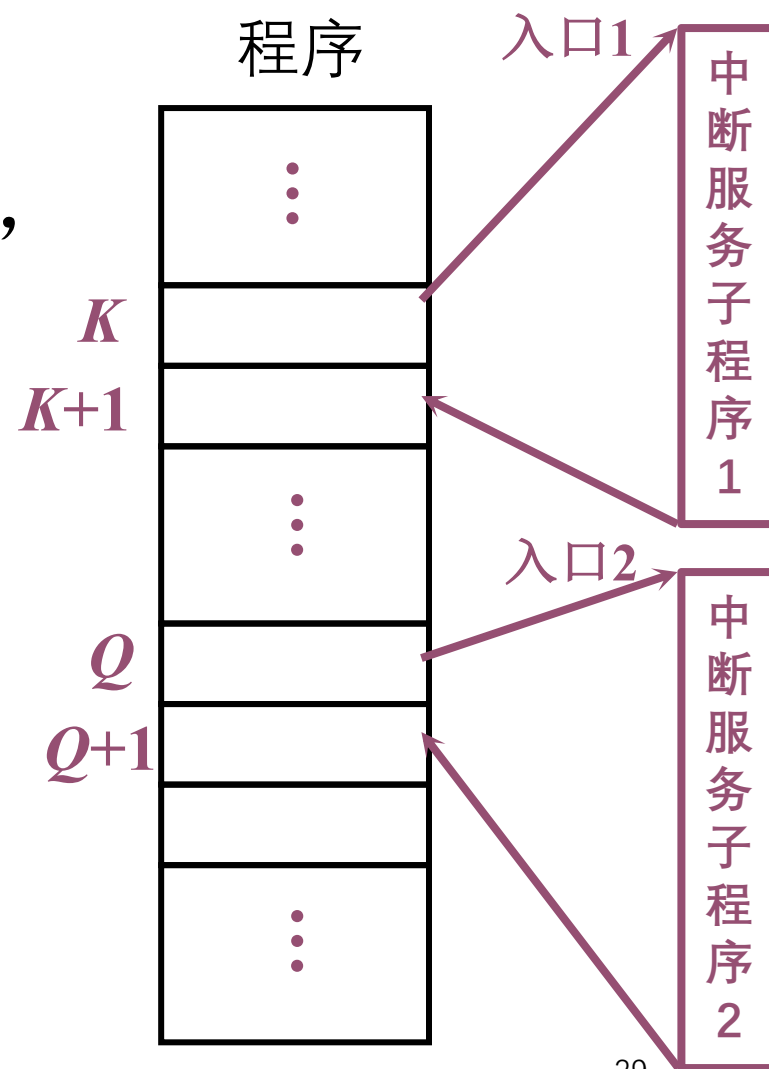


程序中断方式

1. 中断的概念
 2. I/O中断的产生
 3. 中断排队器的实现
 4. I/O中断处理过程
 5. 中断服务程序流程（CPU）
 6. 中断屏蔽技术（CPU）
- 唐书5.5（I/O）、8.4（CPU）

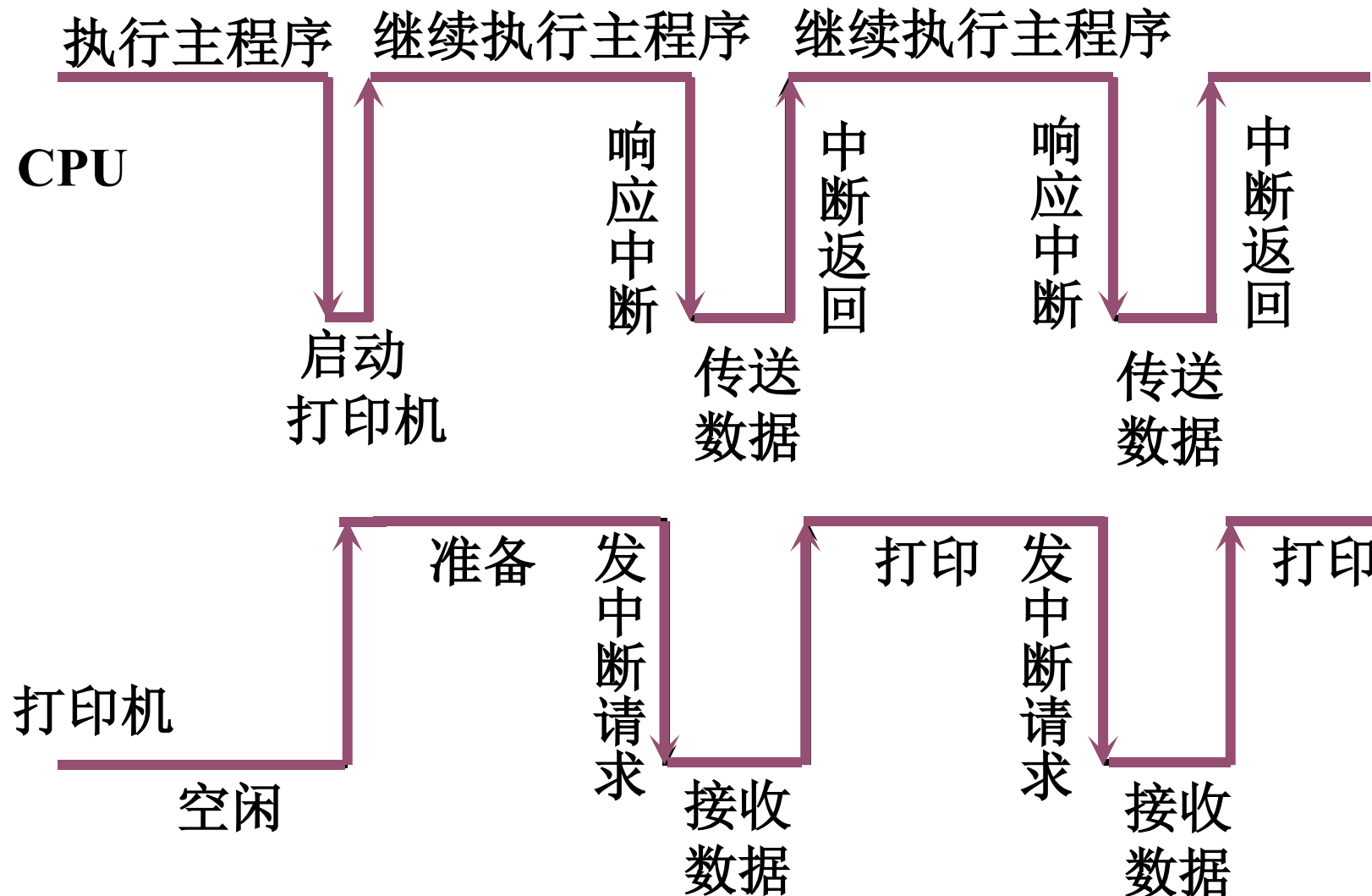
中断的概念

- 中断：CPU暂时中止现程序的执行，转去执行为某个**随机事件**服务的中断服务子程序 (Interrupt Service Routine)，处理完后自动恢复原程序的执行
- 实现主机和外设准备阶段的并行工作
- 避免重复查询外设状态、提升工作效率



I/O中断的产生——以打印机为例

CPU 与打印机并行工作



引起中断的各种因素

- 人为设置的中断
- 程序异常 溢出、操作码不能识别、除法非法
- 硬件故障
- I/O 设备
- 外部事件 用键盘中断 现行程序

I/O设备中断—显卡



程序中中断处理示意图

- 子程序与中断服务子程序的区别？

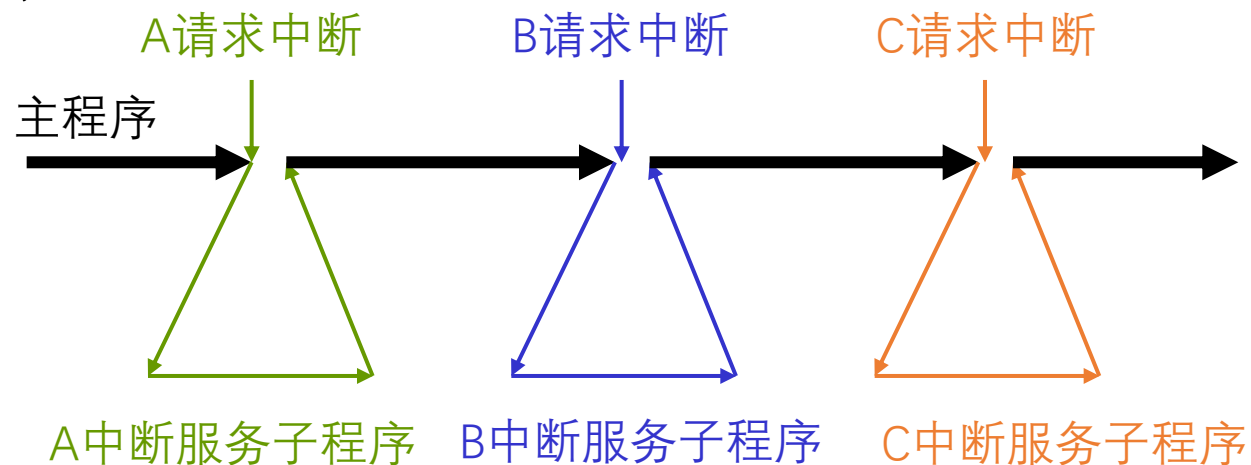
- 子程序在特定位置显式调用
- 中断服务子程序**随机**调用

- 如果A，B，C同时产生中断？

- 中断优先级问题，中断仲裁

- 如果正在运行A中断服务子程序，又收到B中断？

- 中断嵌套



中断系统需解决的问题

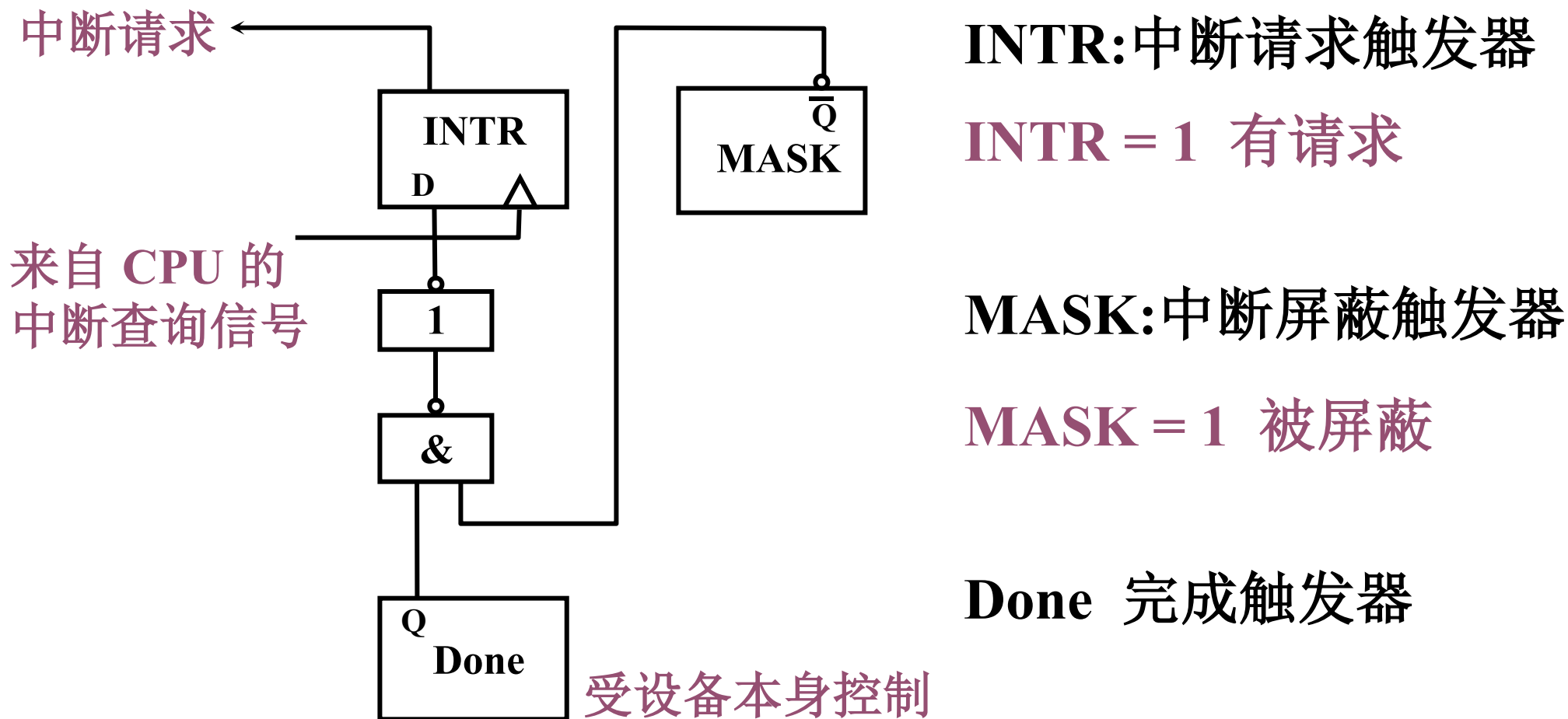
- ① 各中断源 如何 向 CPU 提出请求？
- ② 各中断源 同时 提出 请求 怎么办？
- ③ CPU 什么 条件、什么 时间、以什么 方式响应中断？
- ④ 如何 保护现场？
- ⑤ 如何 寻找入口地址？
- ⑥ 如何 恢复现场，如何 返回？
- ⑦ 处理中断的过程中又 出现新的中断 怎么办？
- ⑧ 硬件 + 软件

程序中中断方式的实现

- 配置中断请求触发器和中断屏蔽触发器
- （中断）排队器——中断判优（仲裁）逻辑
 - 硬件实现（IO接口、CPU）、软件实现
- 中断向量地址形成部件
- 程序中中断方式接口电路的基本组成

程序中中断方式——配置触发器

- 配置中断请求触发器和中断屏蔽触发器



中断请求（INTR）触发器

- 一个中断请求源对应一个 INTR 中断请求标记触发器
 - INTR 分散 在各个中断源的 接口电路中
 - INTR 集中在 CPU 的中断系统内
- 多个 INTR 触发器组成中断请求寄存器

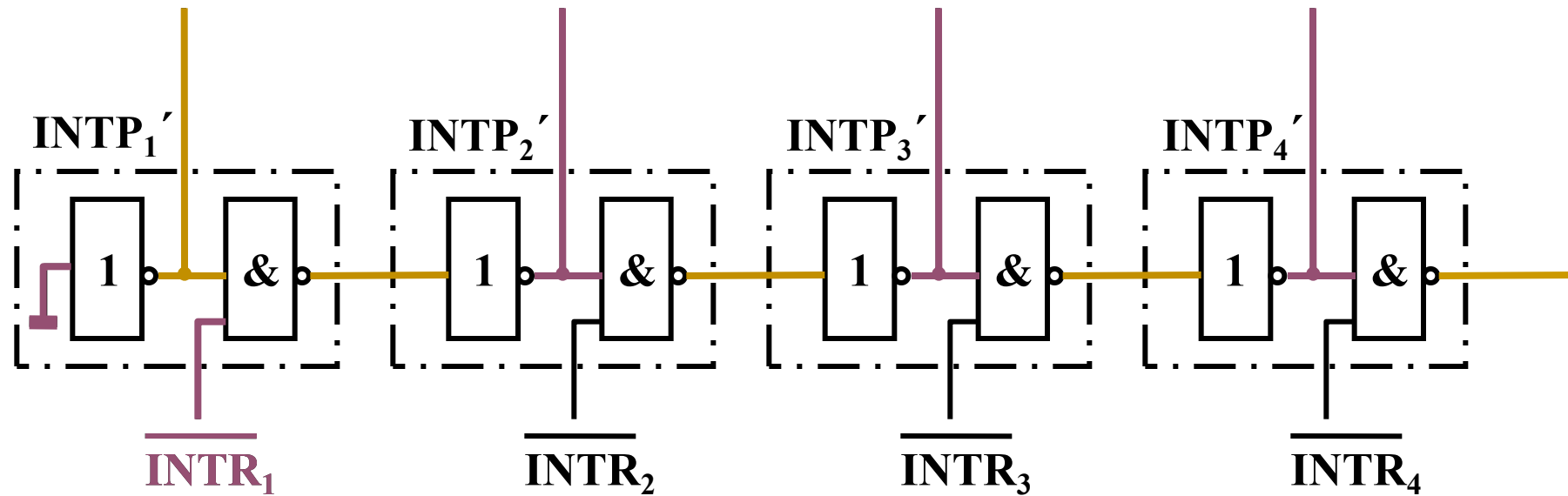
1	2	3	4	5			<i>n</i>
掉电	过热	主存读写校验错	阶上溢	非法除法		键盘输入	打印机输出

划分优先级的常用方法

- 硬件故障中断属于最高级，其次是程序错误中断
- 非屏蔽中断优于可屏蔽中断
- DMA请求优先于I/O设备传送的中断请求
- 高速设备优于低速设备
- 输入设备的中断优于输出设备
- 实时设备优先于普通设备

接口电路中的（中断）排队器

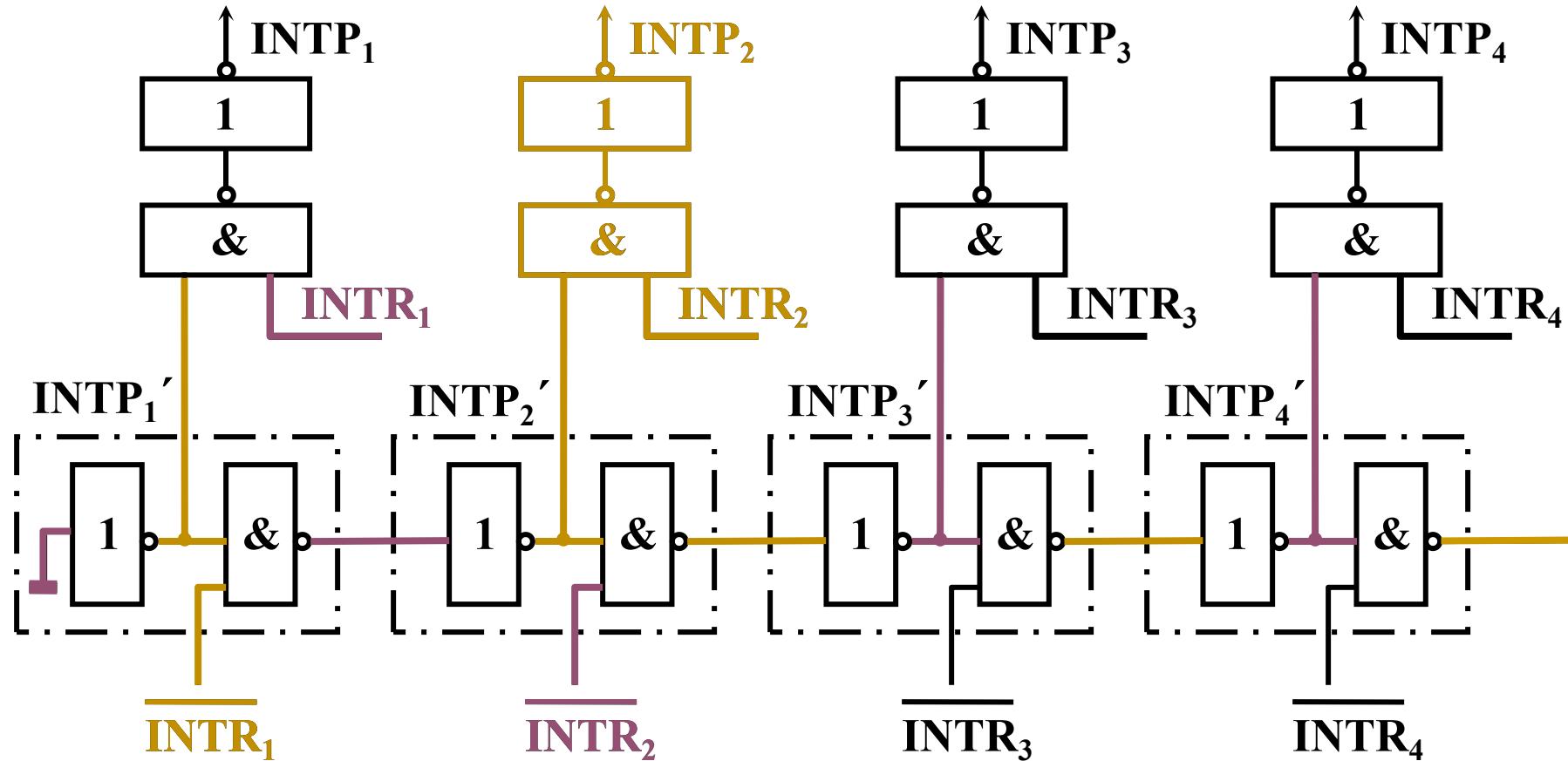
排队 { 硬件 在 CPU 内或在接口电路中（链式排队器）
软件



$INTR_i = 1$ 有请求 即 $\overline{INTR_i} = 0$

设备 1#、2#、3#、4# 优先级按 降序排列

接口电路中的（中断）排队器硬件实现

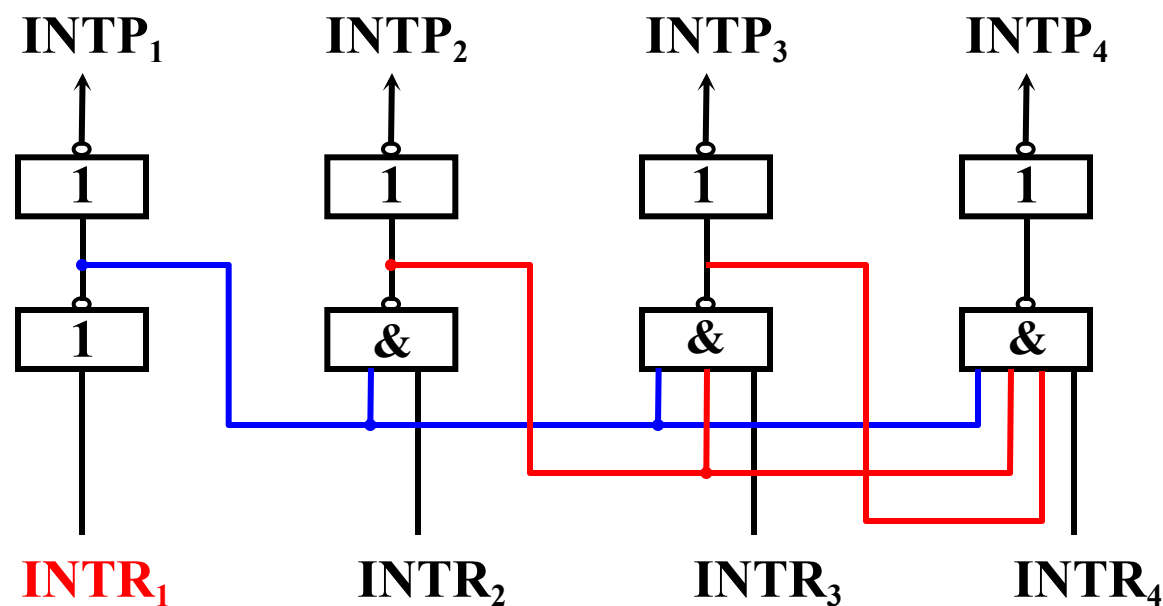


排队器的硬件实现（CPU端）

1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中——链式排队器

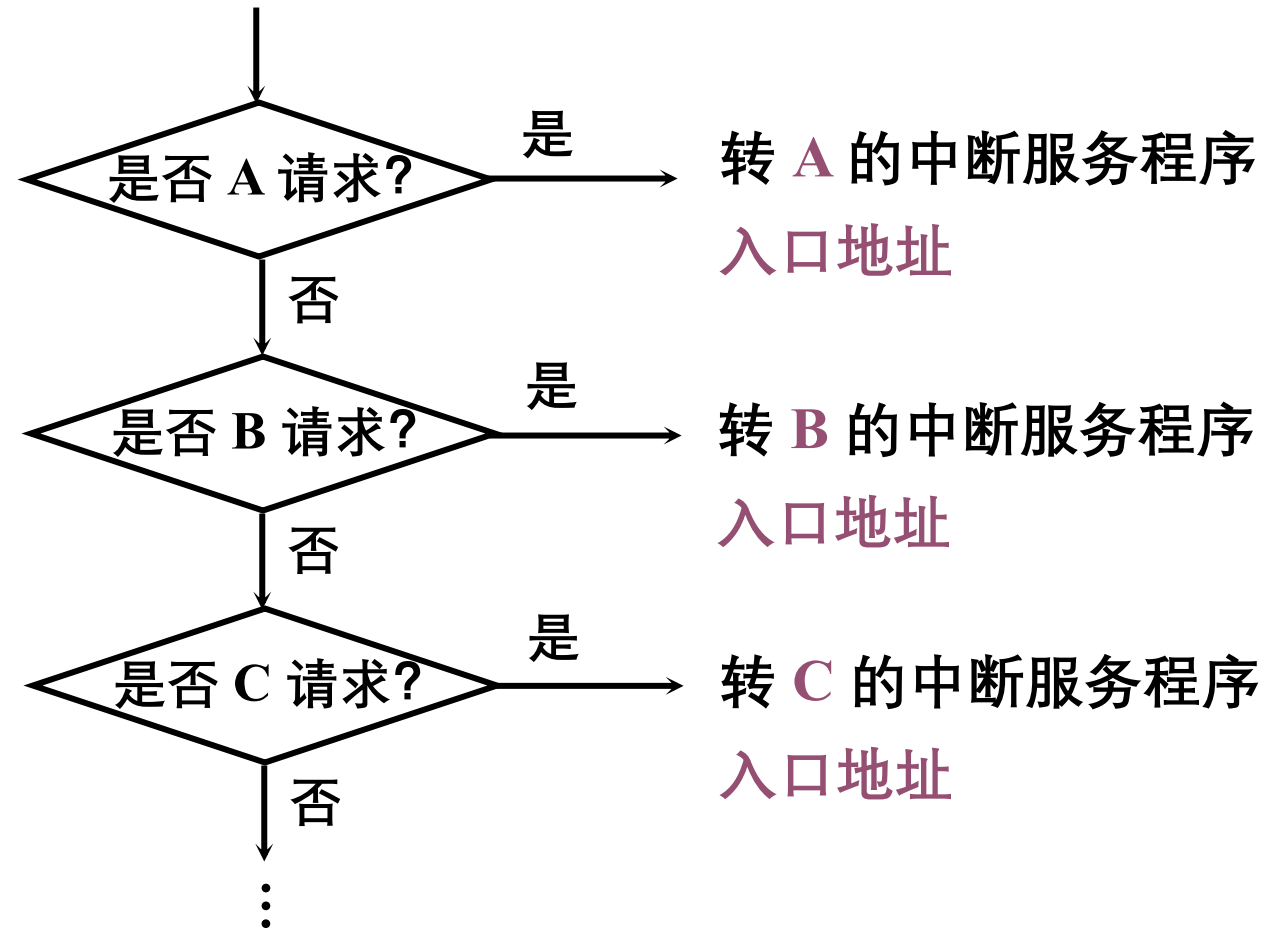
② 集中在 CPU 内



$INTR_1$ 、 $INTR_2$ 、 $INTR_3$ 、 $INTR_4$ 优先级按降序排列

（中断）排队器——软件实现（程序查询）

假设A、B、C 优先级按 降序 排列



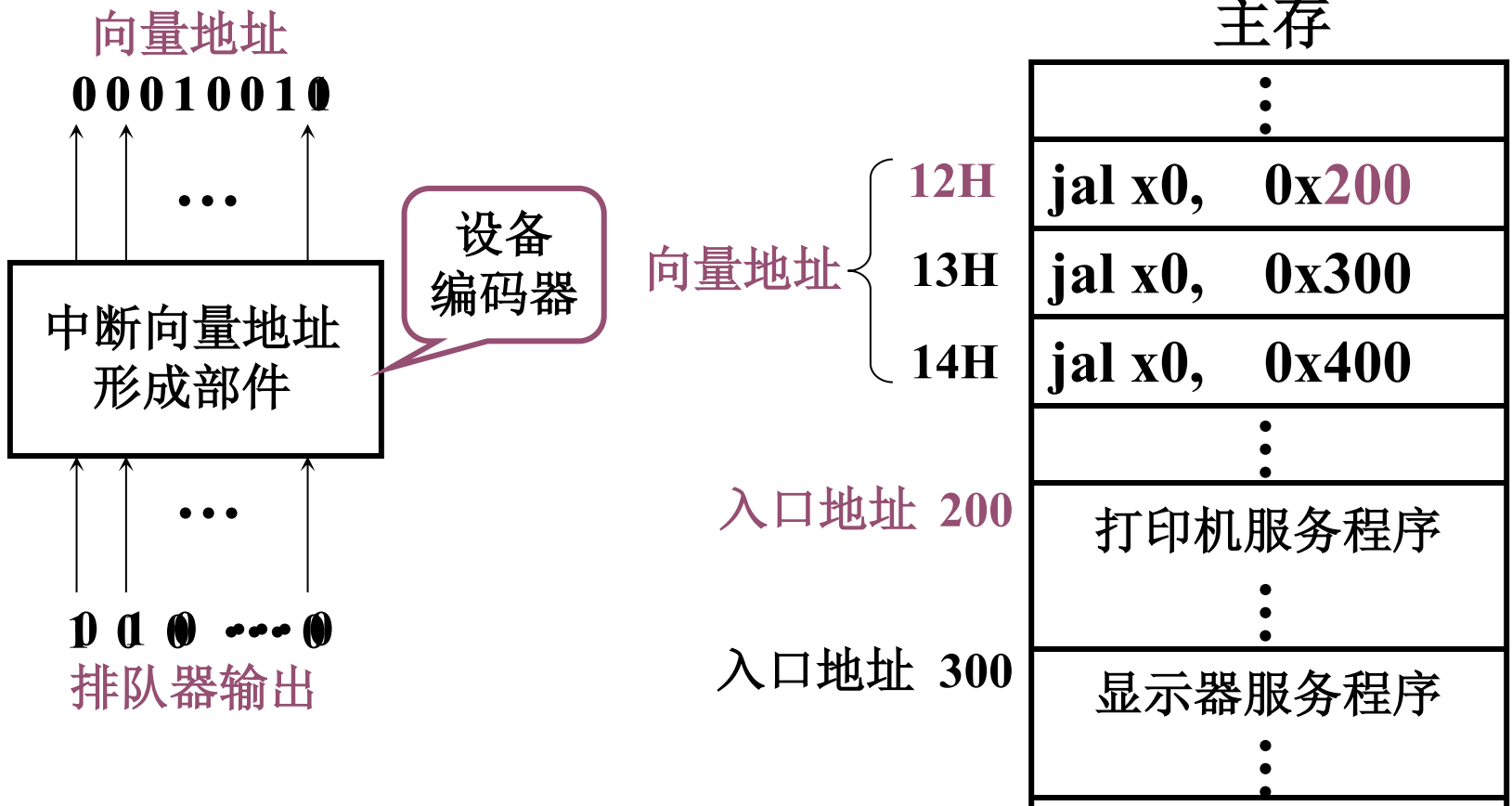
中断向量地址形成部件

中断服务程序入口地址

由软件产生
硬件向量法

由 **硬件** 产生 **向量地址**

再由 **向量地址** 找到 **入口地址**
主存



中断服务程序入口地址——软件查询法

八个中断源 1, 2, ... 8 按 **降序** 排列

假设中断状态寄存器地址为0x1000

每个中断源的完成触发器D对应寄存器的一位

位7对应中断源1, 位0对应中断源8

中断服务程序入口地址

1_SR1: # 中断1服务程序入口

...

2_SR2: # 中断2服务程序入口

...

... 其他中断服务程序

入口地址M:

加载中断状态寄存器

lui t0, 0x1 # 加载高位地址

lw t1, 0x0(t0) # 读取中断状态

检查中断源1 (位7)

andi t2, t1, 0x80 # 检查第7位

beq t2, zero, check_2 # 如果D=0, 检查下一个

jal zero, 1_SR1 # 跳转到中断1服务程序

检查中断源2 (位6)

check_2: andi t2, t1, 0x40

beq t2, zero, check_3

jal zero, 2_SR2

...

no_intr: # 无中断, 返回

jalr zero, ra, 0

中断响应

- CPU 响应中断的条件

- 开中断: 允许中断触发器 $EINT = 1$ (Enable Interrupt)

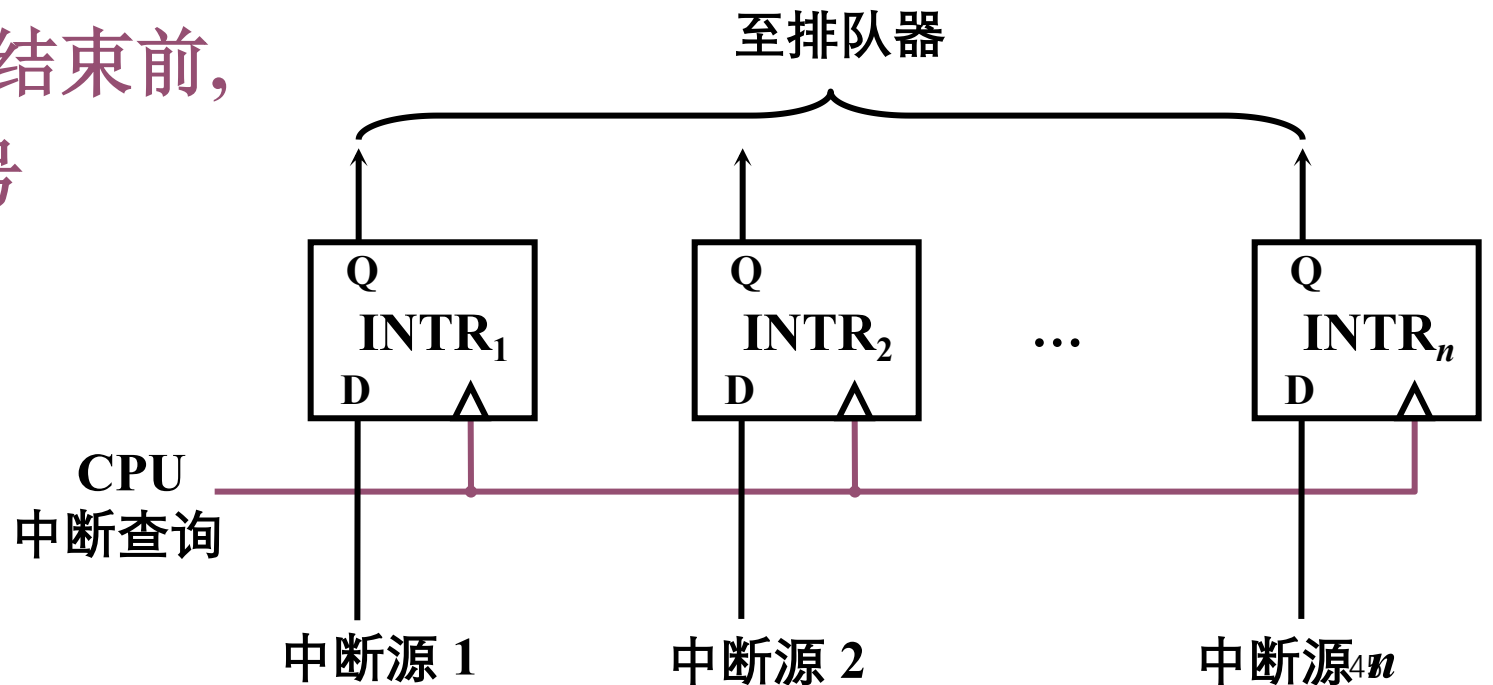
- CPU 响应中断的时间

当 $D = 1$ (随机) 且 $MASK = 0$ 时,

在每条指令执行阶段结束前,

CPU 发 中断查询信号

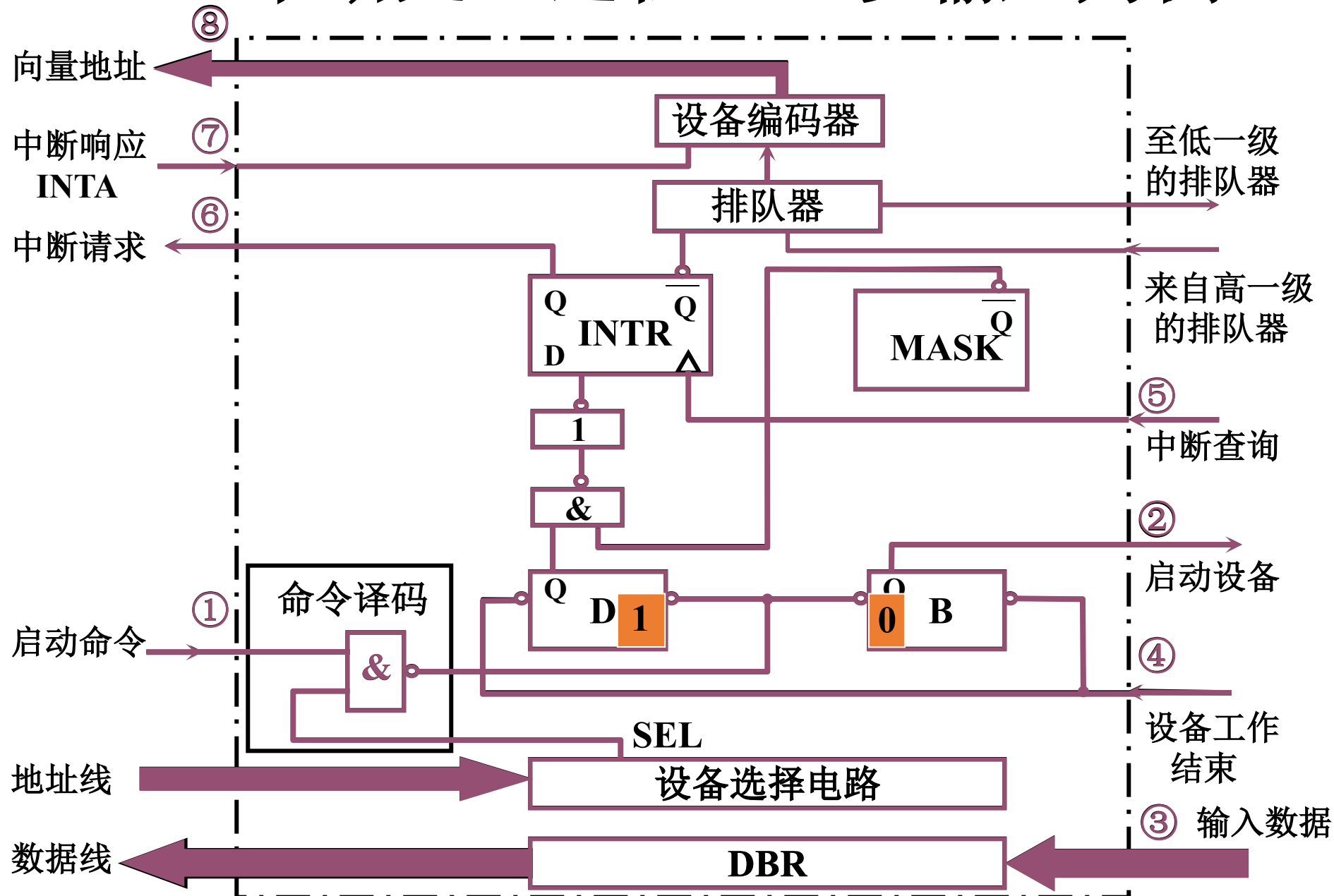
(将 INTR 置 “1”)



中断“隐指令”

- 保护程序断点
 - 断点存于 **特定地址** 内（断点进栈）
- 寻找服务程序入口地址
 - **向量地址**——>PC（硬件向量法）
 - **中断识别程序** 入口地址M——>PC（软件查询法）
- 硬件**关中断**

I/O 中断处理过程——以输入为例



单重中断和多重中断

- 单重中断不允许中断现行的中断服务程序

- 多重中断的概念

- 允许级别更高的中断源

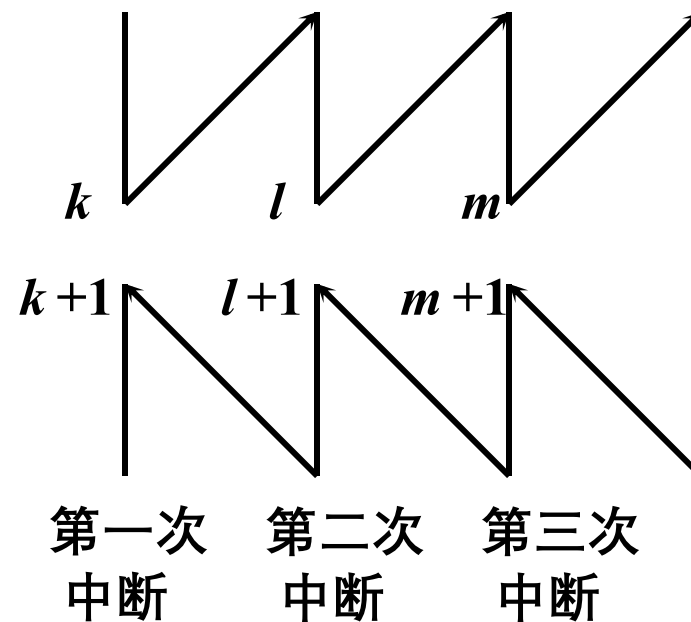
中断现行的中断服务程序

- 实现多重中断的条件:

- 提前 设置 开中断 指令

- 优先级别高的中断源 有权中断

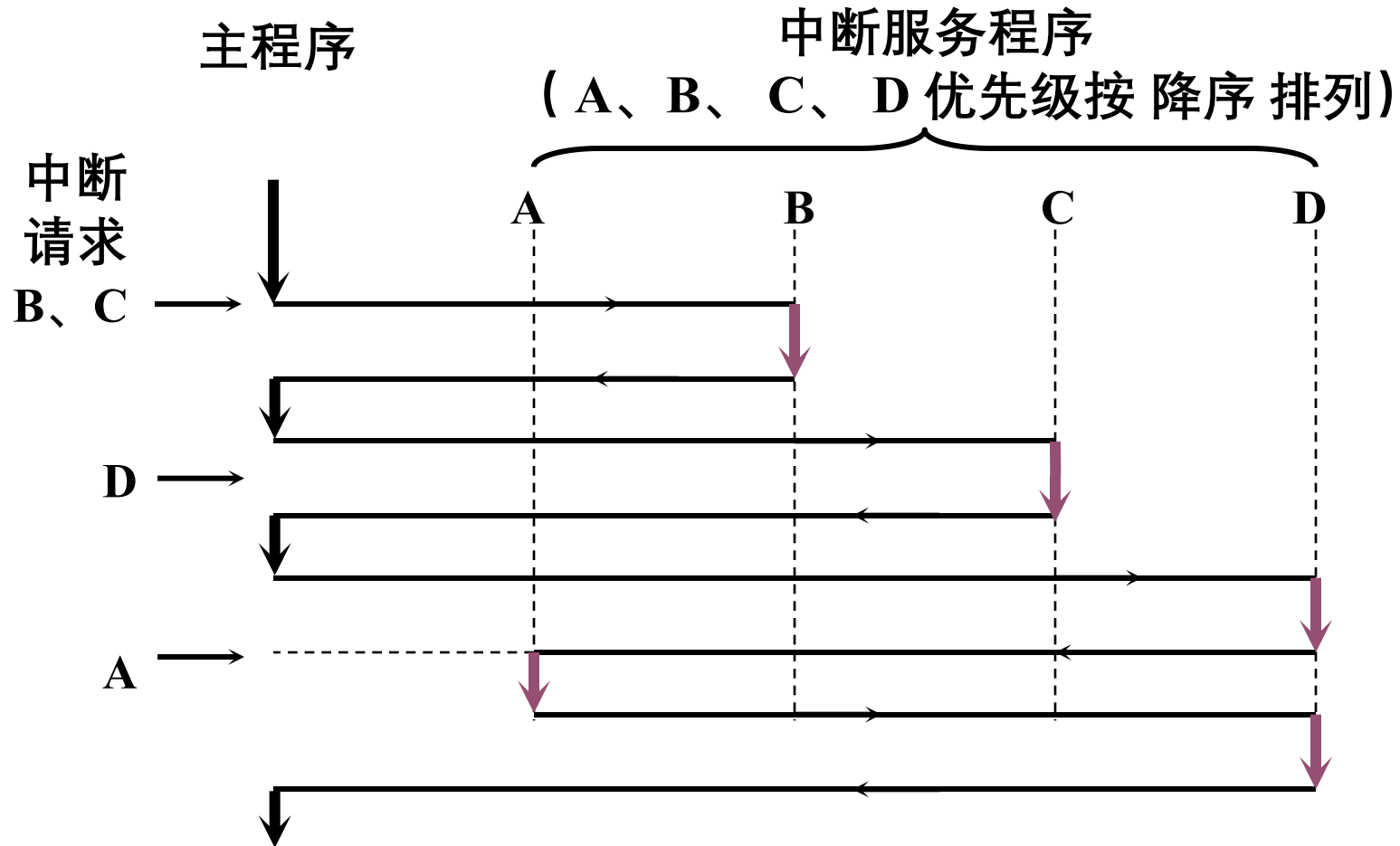
优先级别低 的中断源



程序断点 $k+1$, $l+1$, $m+1$

多重中断示意图

- 1) 提前 设置 开中断 指令
- 2) 优先级别高 的中断源 有权中断优先级别低 的中断源





中断屏蔽技术——屏蔽字

- 16个中断源 1, 2, 3, ..., 16 按 降序 排列

优先级	屏蔽字															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
5	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
⋮	⋮															
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

中断屏蔽可改变**处理**优先等级

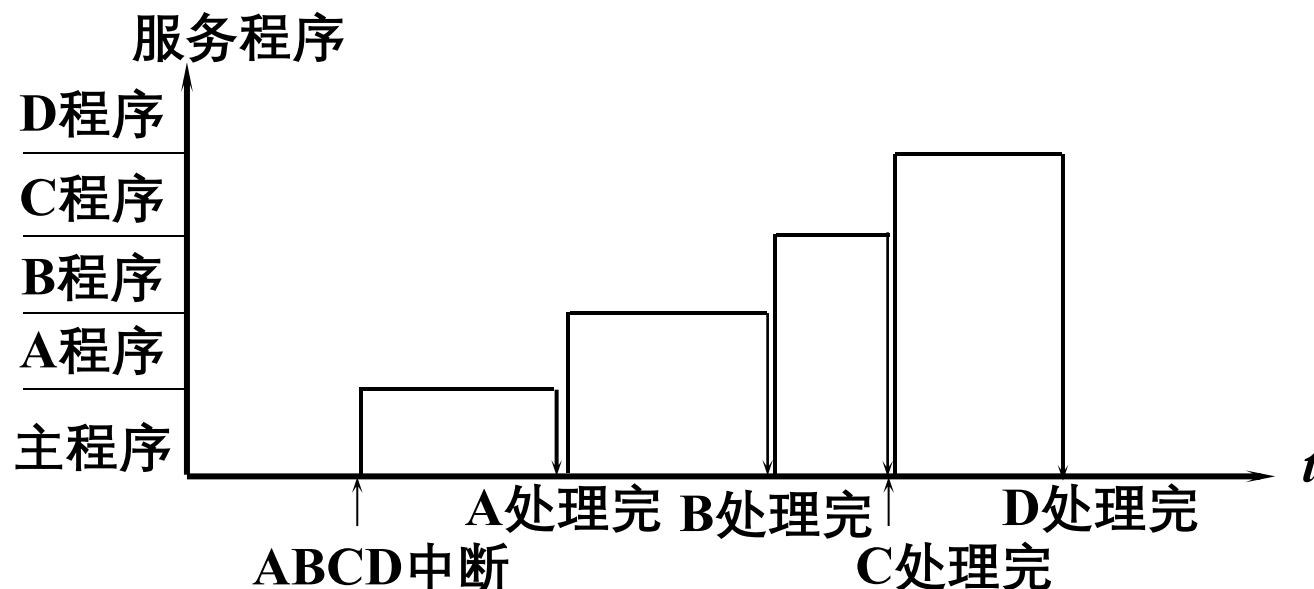
- 响应优先级 **不可改变**
- 处理优先级 可改变（中断处理时，重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

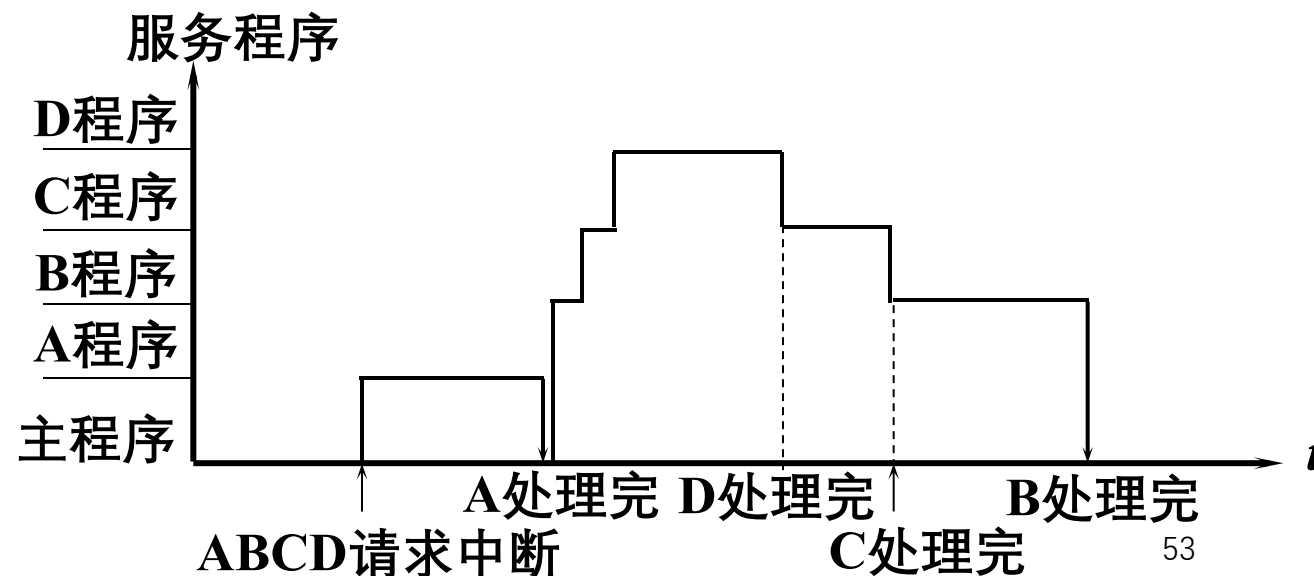
- 响应优先级 **A→B→C→D** 降序排列
- 处理优先级 **A→D→C→B** 降序排列
- 当1代表屏蔽中断时，可以根据新屏蔽字构成的二进制数大小来判断处理优先级。

改变中断处理优先级示意图

- CPU 执行过程（原屏蔽字）
- 优先级 $A \rightarrow B \rightarrow C \rightarrow D$ 降序排列

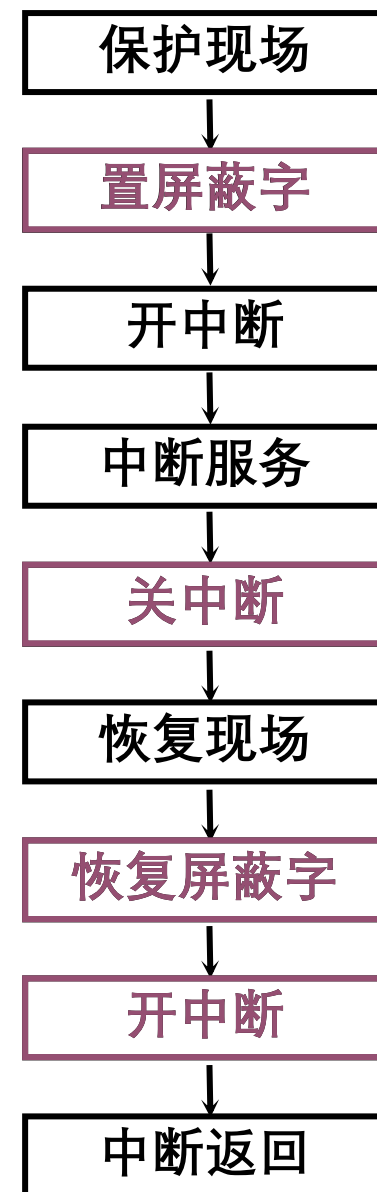


- CPU 执行过程（新屏蔽字）
- 优先级 $A \rightarrow D \rightarrow C \rightarrow B$ 降序排列



屏蔽技术的其他作用

- 可以 人为地屏蔽 某个中断源的请求
- 便于程序控制
- 新屏蔽字的设置



(I/O) 中断服务程序流程

1) 保护现场

程序断点的保护——由中断隐指令完成

寄存器内容的保护——由进栈指令完成

2) 中断服务（设备服务）

对不同的 I/O 设备具有不同内容的设备服务

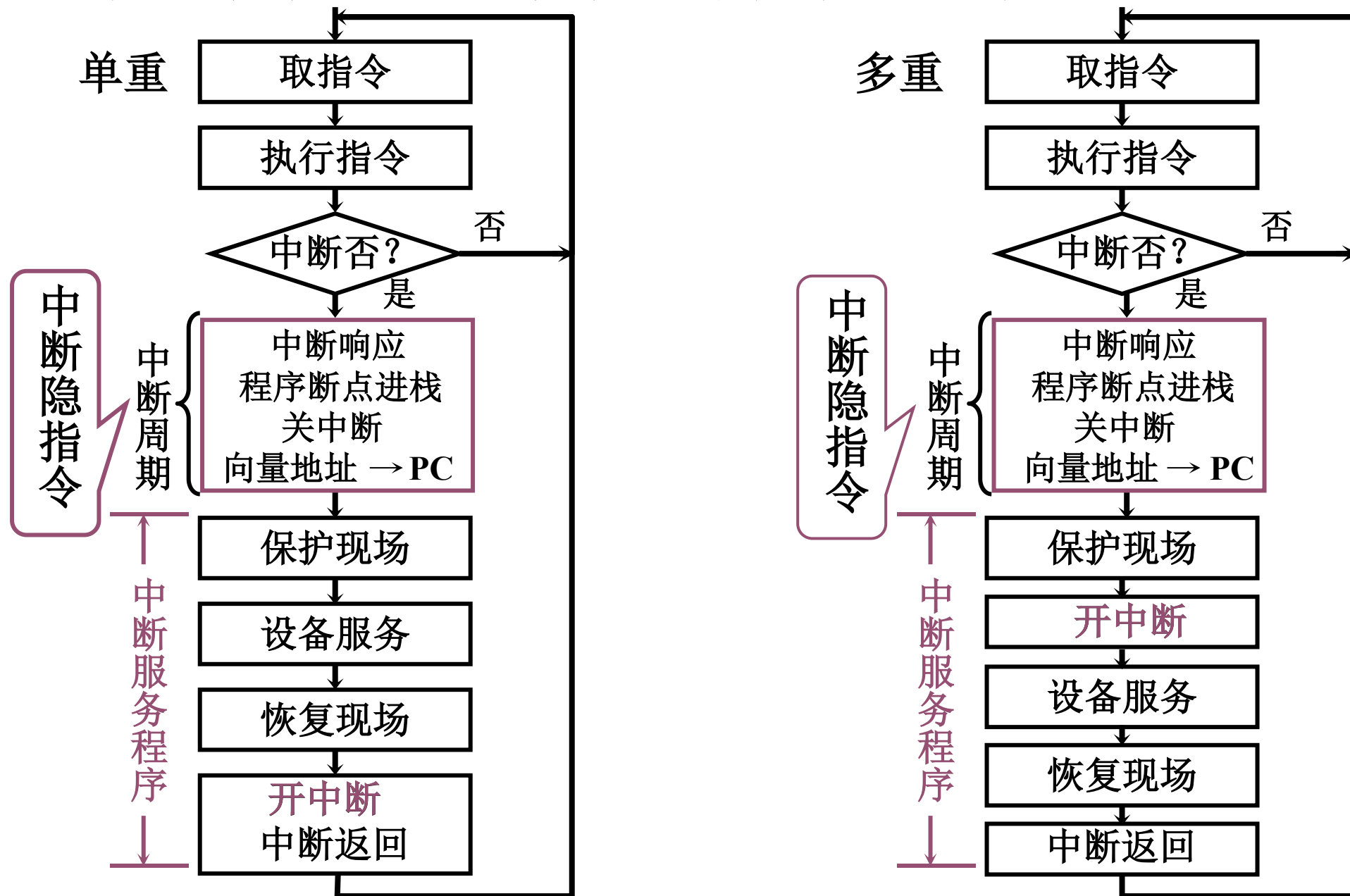
3) 恢复现场

出栈指令

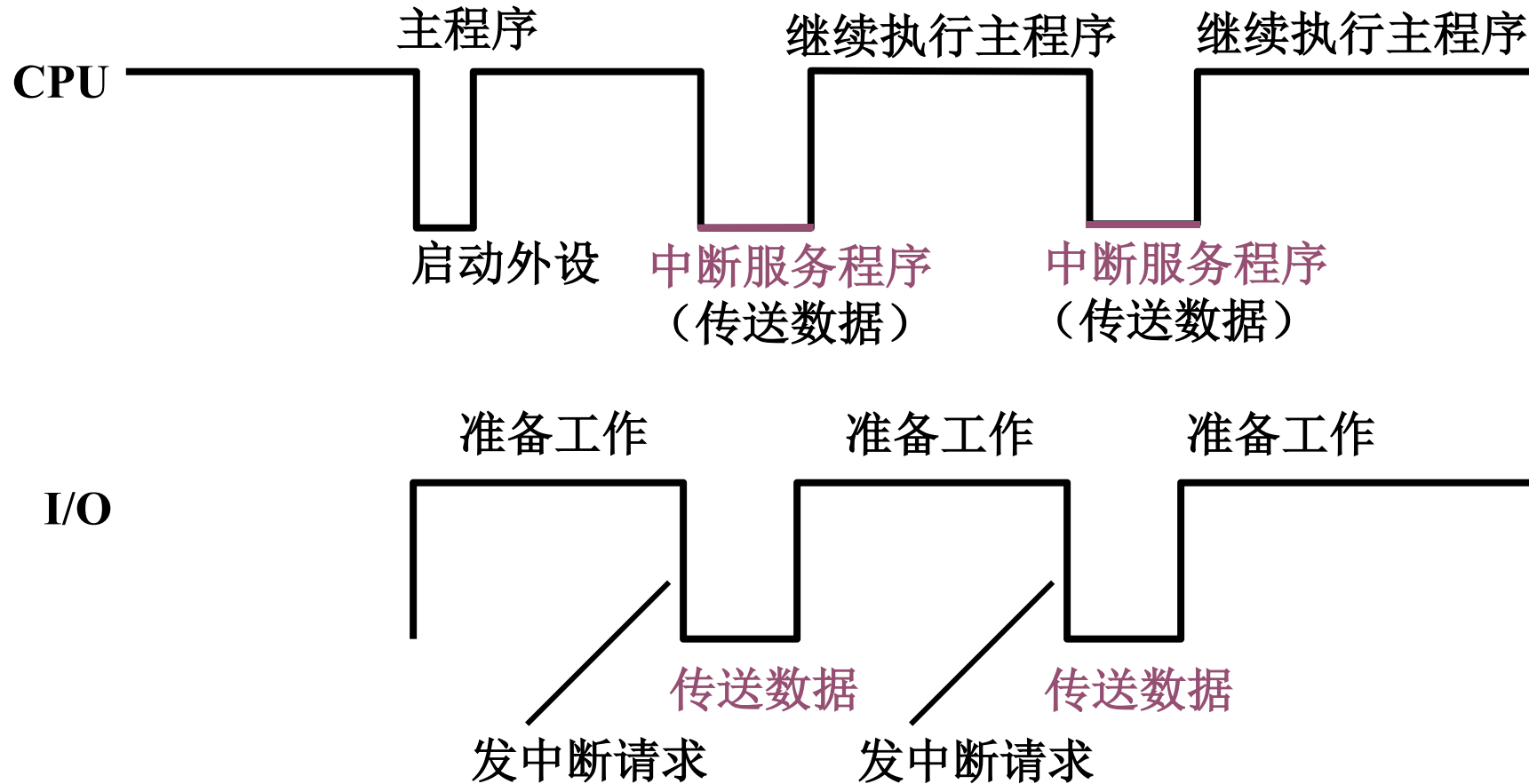
4) 中断返回

中断返回指令

单重中断和多重中断的服务程序流程



主程序和中断服务程序抢占CPU示意图



宏观上 CPU 和 I/O 并行工作

微观上 CPU 中断现行程序为 I/O 服务

第七章 输入输出系统

一. 概述

二. I/O接口

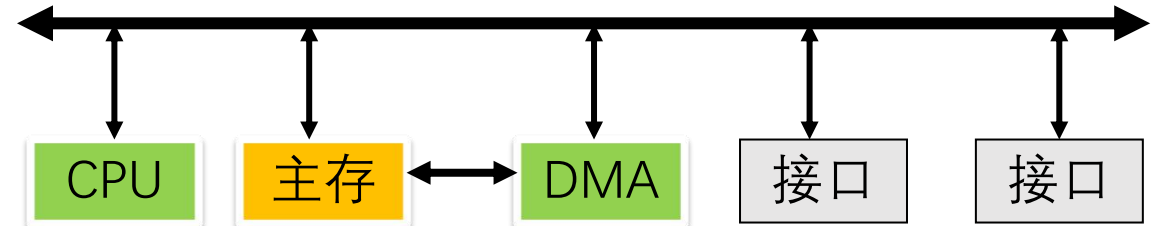
三. 程序查询方式

四. 程序中断方式

五. **DMA方式**

直接内存访问(DMA:Direct Memory Access)

- 中断方式
 - 传送一个数据执行一次中断服务子程序（几十条指令）
 - 频繁占用CPU，效率低下
 - 不适合于成组数据交换、高速传输的系统
- DMA方式
 - 外设与主存间建立一个由**硬件**管理的数据通路
 - CPU不介入外设与主存的数据传送操作
 - 减少CPU开销，提升效率
 - 适合成组交换数据的场合

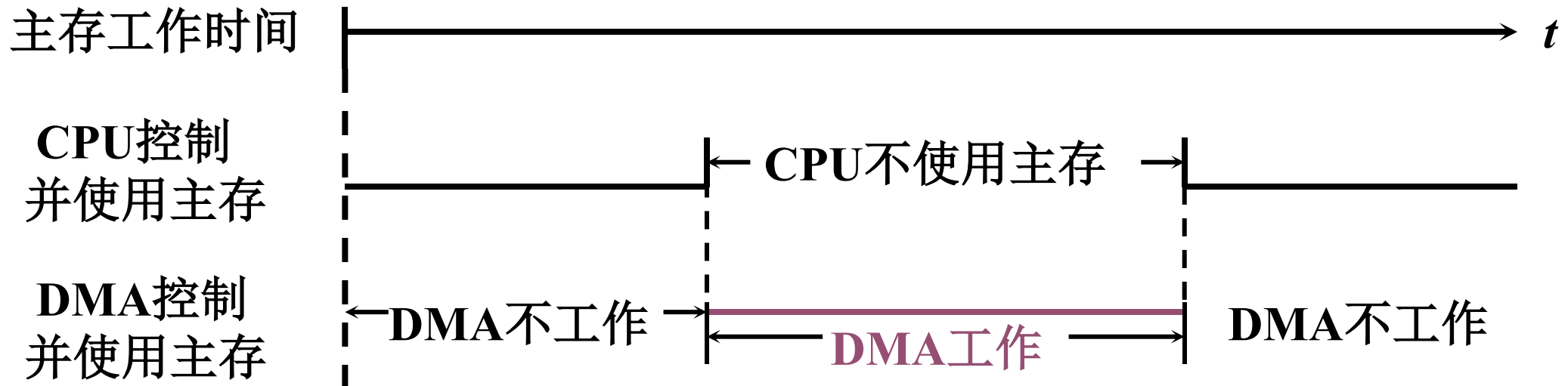


内存争用

- DMA方式进行数据传送时
 - DMA控制器直接访问内存
 - CPU执行主程序（需要访存）
 - 主存使用权的冲突（资源冲突）
- 如何避免内存争用？
 - 停止CPU使用主存
 - DMA与CPU交替使用主存
 - 周期挪用（周期窃取）

解决内存争用方法1——停止CPU访问主存

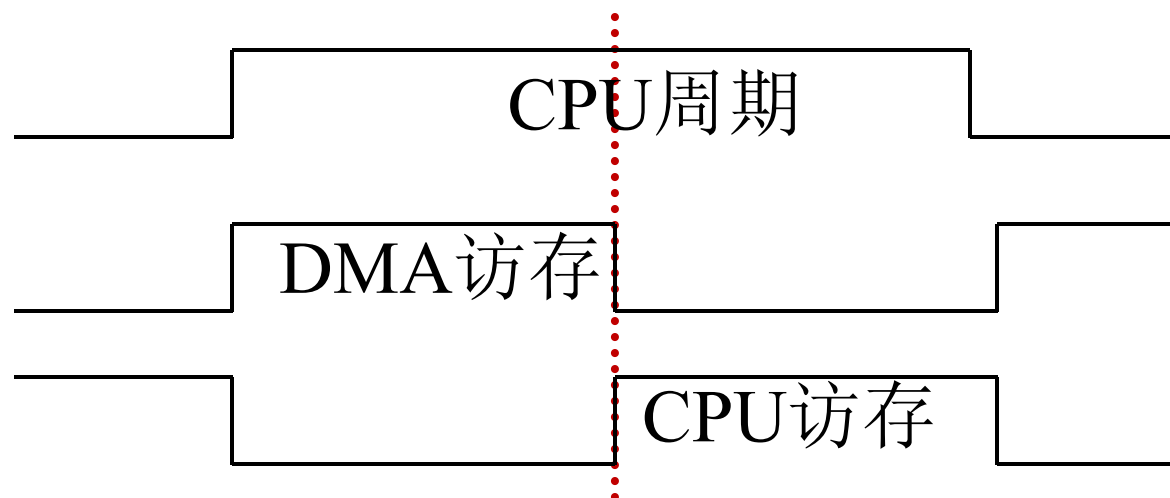
- 控制简单
- 未充分发挥 CPU 对主存的利用率
- 若DMA批量数据传输周期过长，CPU长期无法访问内存
- 若外设传送两个数据的时间间隔大于存储周期，内存未充分利用



解决内存争用方法2—DMA与CPU交替访问

CPU 工作周期 { 前半周期 C_1 专供 DMA 访存
后半周期 C_2 专供 CPU 访存

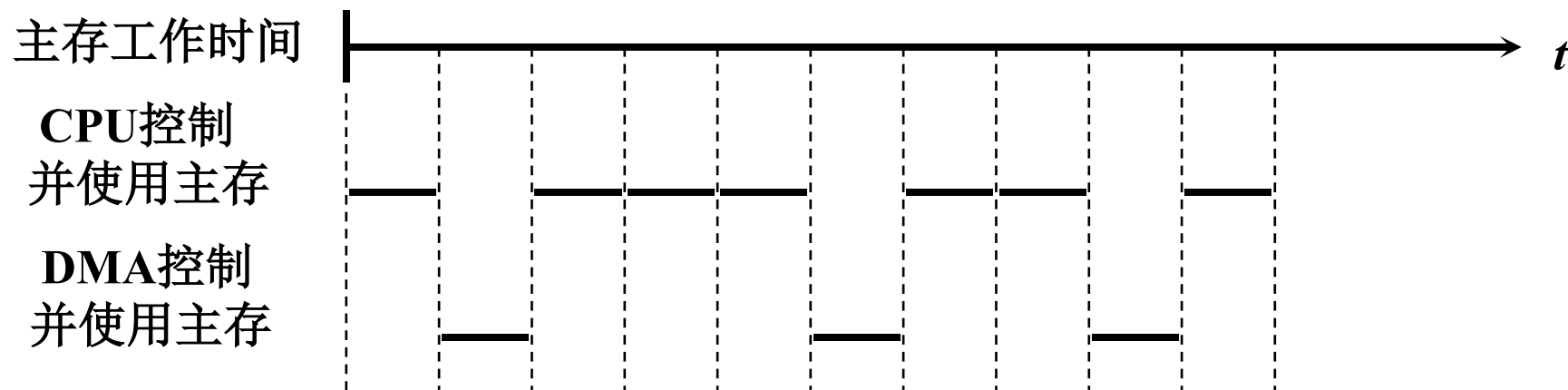
所有指令执行过程中的基准时间



不需要 申请和归还 总线的使用权

解决内存争用方法3—周期挪用

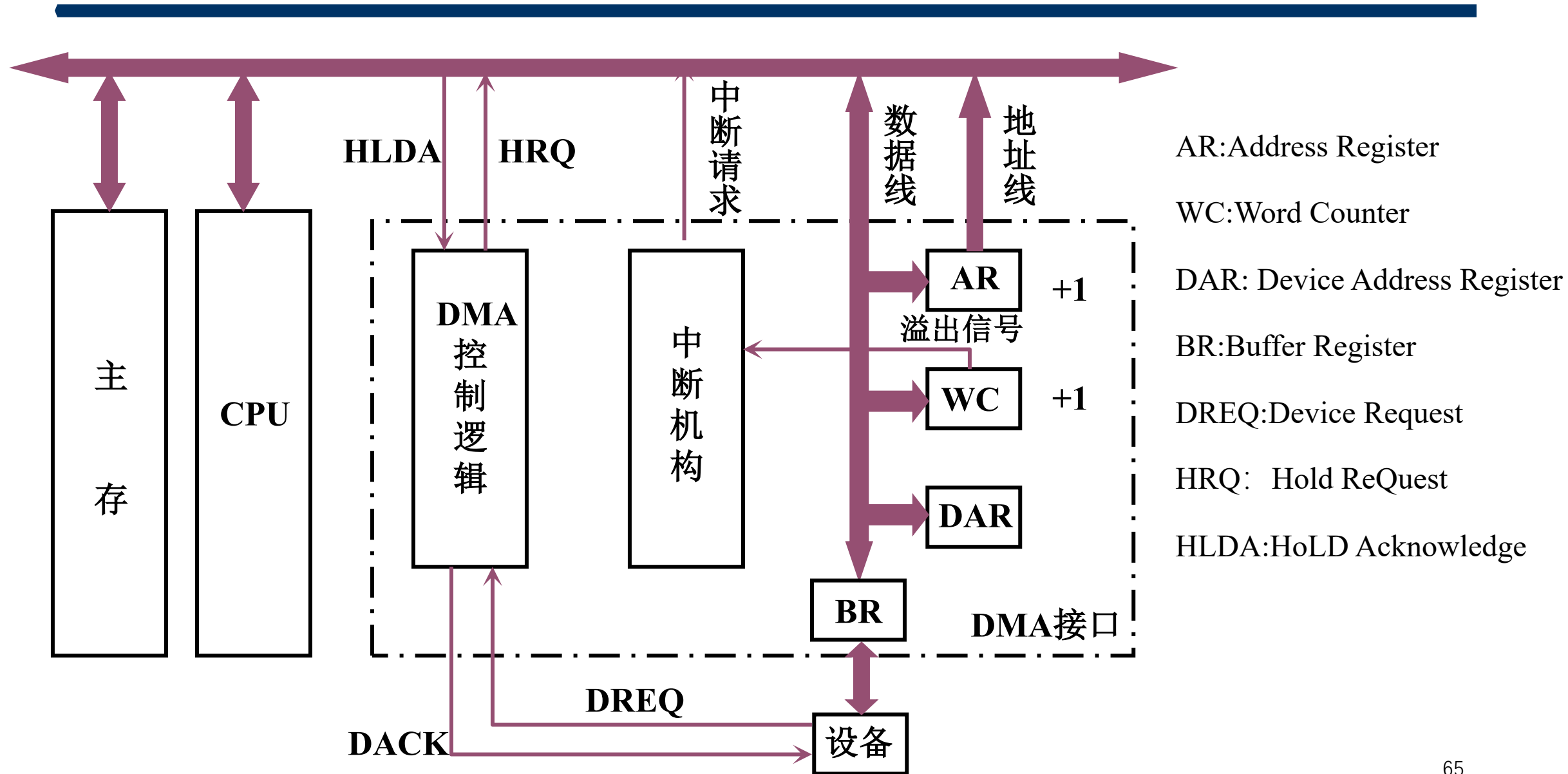
- DMA要求访存时，CPU暂停一个或多个**存储周期**，将**总线控制权**让给 **DMA**，**数据**传送结束后，CPU继续运行。
- CPU现场没有变动，仅延缓了指令的执行
 - **周期挪用**或称**周期窃取**。



DMA 接口的功能

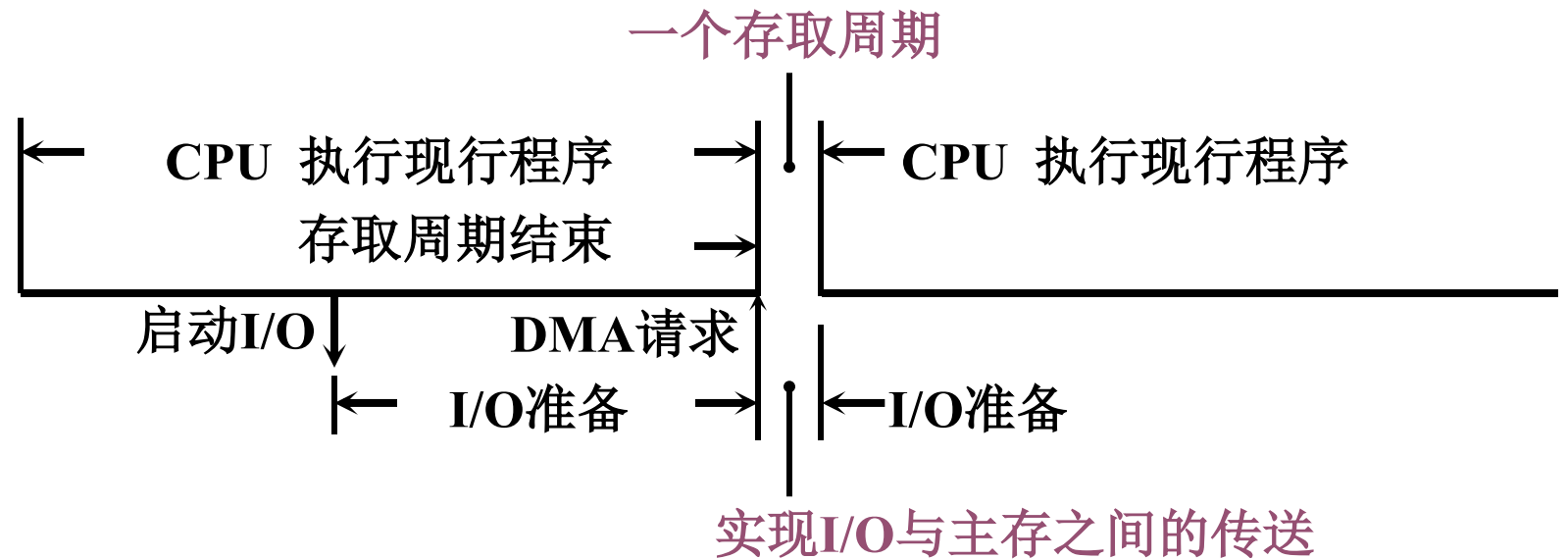
- 向 CPU 申请 DMA 传送
- 处理 总线控制权的转交
- 管理 系统总线、控制 数据传送
- 确定 数据传送的 首地址和长度
 - 更新 传送过程中的数据 地址 和 长度
- DMA 传送结束时， 给出操作完成信号

DMA 接口的组成



I/O设备与主机信息传送——DMA方式

- 主存和I/O之间直接交换数据
- 不中断现行程序
- 周期挪用（周期窃取）
- CPU 和 I/O 并行工作



DMA的工作过程

1. 预处理（准备阶段）

主机通过CPU指令向DMA接口发送的传输参数，并启动DMA

- 数据传输的方向
- 数据块在主存的首地址
- 数据在外设存储介质上的地址
- 数据的传输量

2. 数据传输传送阶段

3. 后处理（结束阶段）

DMA 传输过程示意

CPU

预处理:

主存起始地址 → DMA
设备地址 → DMA
传送数据个数 → DMA
启动设备

数据传送:

继续执行主程序
同时完成一批数据传送

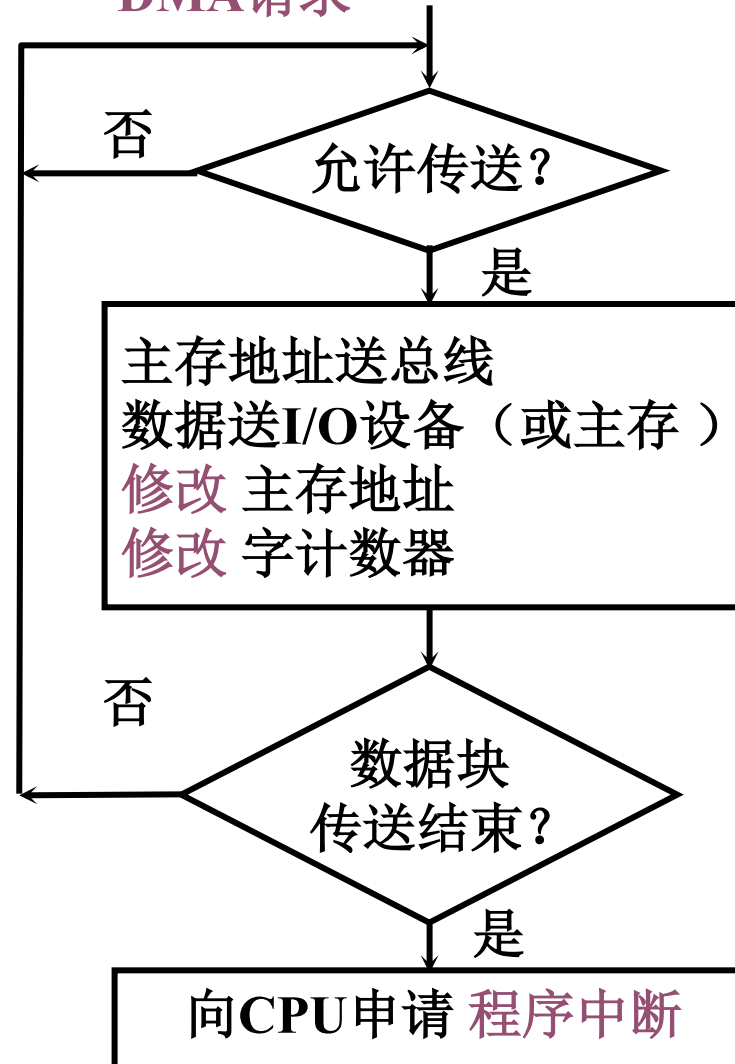
后处理:

中断服务程序
做 DMA 结束处理

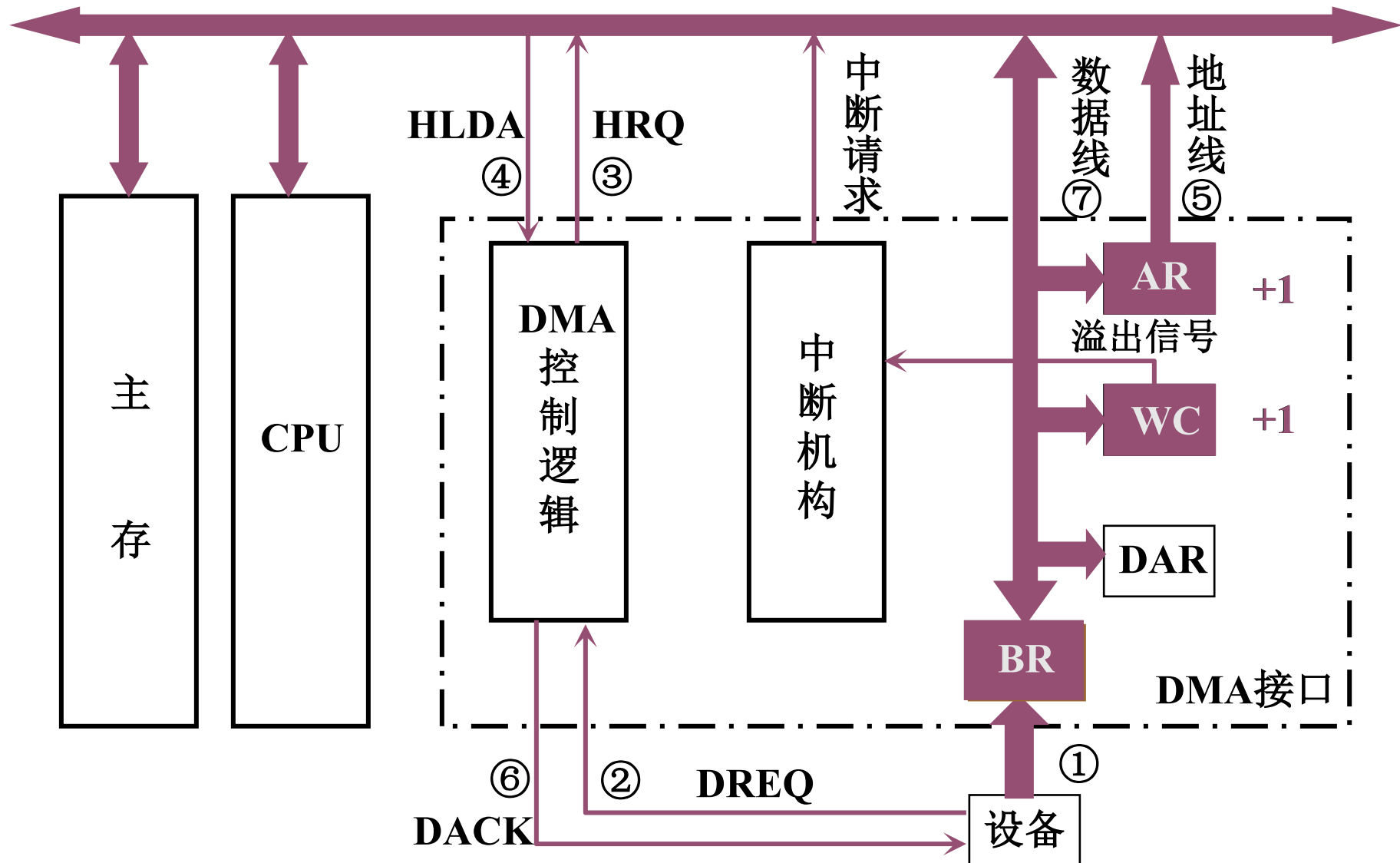
继续执行主程序

数据传送

DMA请求



数据传送过程（输入）



DMA工作过程——后处理

- 校验送入主存的数是否正确
- 是否继续用 DMA
- 测试传送过程是否正确，错则转诊断程序
- 由中断服务程序完成

例：DMA方式与中断方式的对比

- 设字符设备的传输率为 9600 bps，采用周期窃取方式把字符传送到存储器，其最大批量为 400 字节（忽略预处理时间），若存取周期为 100 ns，处理一次中断需 5 μ s，请问：

1)用 DMA 方式每秒因数据传输需占用处理器多少时间？

2)用中断方式，需占处理器多少时间？

解：根据字符设备的传输率 9600 bps，得

每秒能传输 $9600/8 = 1200$ B，即 1200 个字符

DMA方式： $0.1\mu\text{s} \times 1200 + 5\mu\text{s} \times (1200/400) = 135\mu\text{s}$

中断方式： $5\mu\text{s} \times 1200 = 6000\mu\text{s}$

例：DMA数据交换

- 采用DMA方式实现磁盘与主机交换信息，设磁盘传输速率为 2 MBps，平均传输的数据长度为 4 KB，DMA的预处理需 1000 个时钟周期，DMA 的后处理需 500 个时钟周期，若磁盘不断进行传输，请问 50MHz 的处理器需用多少时间比率进行DMA辅助操作（预处理+后处理）？（1M=1024K）

解：每秒所需DMA 辅助操作的时钟周期数

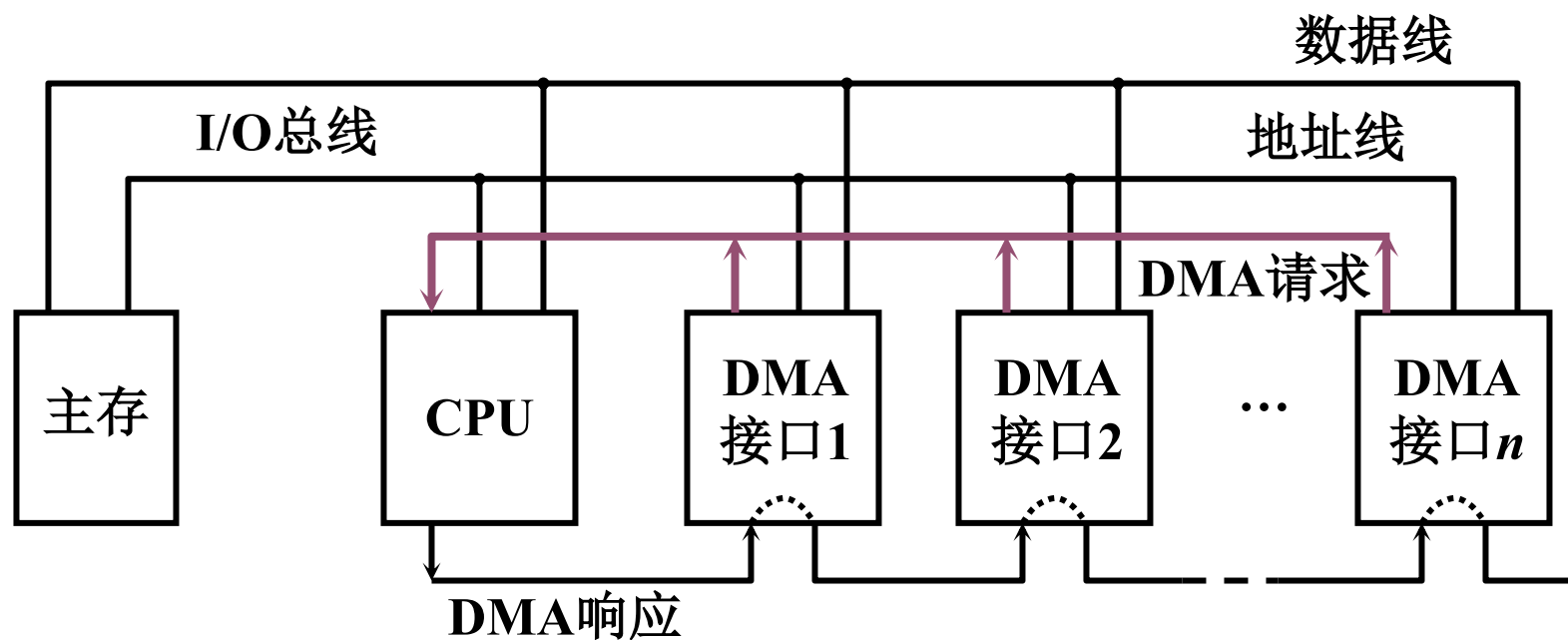
$$(2\text{M}/4\text{K}) \times (1000 + 500) = 512 \times 1500 = 768000$$

故DMA 辅助操作占用 CPU 的时间比率为

$$[768000 / (50 \times 10^6)] \times 100\% = 1.536 \%$$

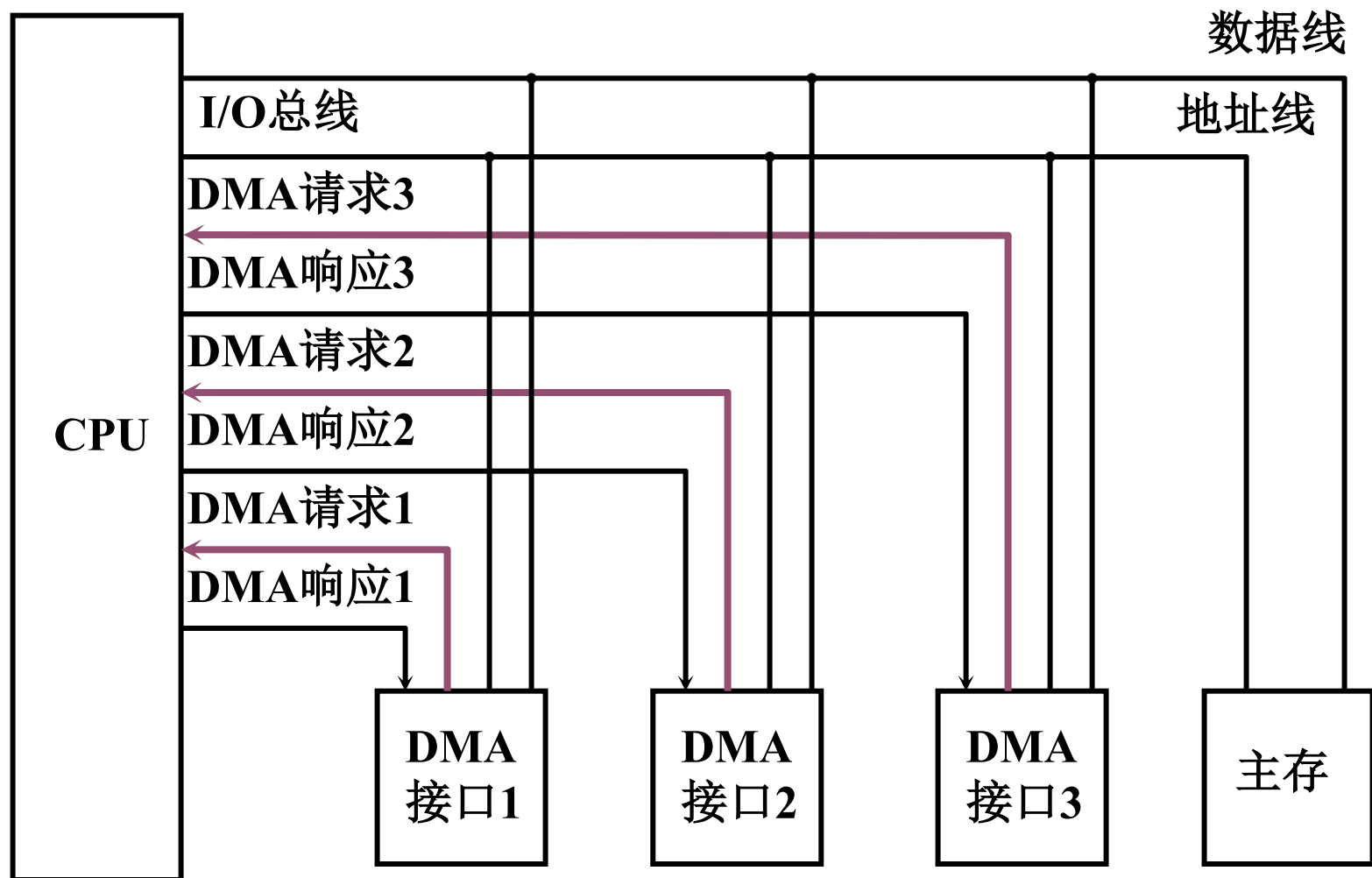
DMA 接口与系统的连接方式

1) 具有公共请求线的 DMA 请求



DMA 接口与系统的连接方式

2)独立的DMA请求



DMA方式与程序中断方式的比较

	中断方式	DMA方式
• 数据传送	程序	硬件
• 响应时机	指令周期后期	存储周期后期
• 处理异常情况	能	不能
• 中断请求	传输数据	后处理
• 优先级	低	高

三种方式的CPU工作效率比较

